

INSTITUTO TECNOLÓGICO AUTÓNOMO DE MÉXICO



Diseño, Construcción y Validación de un Modelo de Riesgo Crediticio

TESIS

QUE PARA OBTENER EL TÍTULO DE

LICENCIADO EN MATEMÁTICAS APLICADAS

PRESENTA

SANTIAGO MURIEL VIZCAINO

ASESOR

DR. EDGAR POSSANI ESPINOSA

«Con fundamento en los artículos 21 y 27 de la Ley Federal del Derecho de Autor y como titular de los derechos moral y patrimonial de la obra titulada “**Diseño, Construcción y Validación de un Modelo de Riesgo Crediticio**”, otorgo de manera gratuita y permanente al Instituto Tecnológico Autónomo de México y a la Biblioteca Raúl Baillères Jr., la autorización para que fijen la obra en cualquier medio, incluido el electrónico, y la divulguen entre sus usuarios, profesores, estudiantes o terceras personas, sin que pueda percibir por tal divulgación una contraprestación.»

FECHA

SANTIAGO MURIEL VIZCAINO

*A mis padres,
por su incansable esfuerzo.*

Agradecimientos

Quiero agradecer a ...

Índice general

1. Introducción	1
2. Caso de estudio	5
3. Técnicas y Modelos	13
3.1. Separación del conjunto de datos	14
3.2. Técnicas de preprocesamiento	16
3.2.1. Tratamiento de datos faltantes	17
3.2.2. Técnicas de codificación para variables no numéricas	20
3.2.3. Transformación de variables	27
3.2.4. Selección de variables	33
3.2.5. Tratamiento para datos no balanceados	38
3.3. Algoritmos de clasificación	39
3.3.1. Regresión Logística	41
3.3.2. Regresión Ridge	45
3.3.3. Árbol de Decisión	47
3.3.4. Bosque Aleatorio	49
3.3.5. XGBoost	51
3.3.6. LightGBM	52
3.4. Métricas de evaluación	52



3.4.1.	Contexto del problema	53
3.4.2.	Matriz de confusión y métricas subyacentes . . .	54
3.4.3.	Balance entre <i>precision</i> y <i>recall</i>	56
3.4.4.	Curvas ROC, PRC y métricas subyacentes	57
3.5.	Entrenamiento, validación y monitoreo del modelo . . .	60
3.5.1.	Validación Cruzada	60
3.5.2.	Búsqueda de hiper-parámetros óptimos	61
3.5.3.	Monitoreo y mejoras del modelo	62
4.	Resultados y Discusión	64
4.1.	Definiendo la ruta óptima de preprocesamiento	65
4.1.1.	Tratamiento de datos faltantes	65
4.1.2.	Codificación de variables no numéricas o categóricas	65
4.1.3.	Transformación de variables	68
4.1.4.	Selección de variables	69
4.1.5.	Balanceo de datos	71
4.2.	Entrenamiento y ajuste de modelos	73
4.3.	Analizando los resultados a detalle	77
4.4.	Definiendo un modelo de toma de decisión	83
	Conclusiones	93
	Bibliografía	96
A.	Anexos	102
A.1.	Sobre los datos de estudio	102
A.1.1.	Extracto del diccionario de datos	102
A.1.2.	Análisis Exploratorio de Datos (EDA)	104
A.2.	Coefficiente de correlación	107
A.3.	WOE y IV	107

A.4. SMOTE	109
A.5. Conceptos clave detrás del aprendizaje de máquina . . .	112
A.5.1. Suma residual de cuadrados y mínimos cuadrados	112
A.5.2. Error cuadrático medio	113
A.5.3. Funciones sigmoide	113
A.5.4. Funciones monótonas	114
A.5.5. Funciones convexas	115
A.5.6. Algoritmos de optimización	116
A.6. Detalles de implementación de los algoritmos	117
A.6.1. Regresión Logística	117
A.6.2. Regresión Ridge	118
A.6.3. XGBoost	118
A.6.4. LGBM	119

Capítulo 1

Introducción

“El crédito pareciera una invención moderna, pero en realidad existe desde al menos 5,000 años [13].” El crédito puede ser definido como un acuerdo contractual donde un prestatario recibe una suma de dinero o algo de valor por parte de un prestamista, lo cual se debe devolver en una fecha posterior, generalmente con intereses [9]. El prestamista es quien **presta el dinero** y el prestatario es quien recibe el préstamo. El fundamento básico de esta figura es el cómo asegurar que el prestatario pagará de vuelta. La evaluación y medición de este riesgo o incertidumbre se conoce como riesgo crediticio, entendido como la posibilidad de perder dinero **por que** el prestatario no paga su préstamo [10]. El evento de incumplir en el pago de una deuda se conoce como **default** [11]. De manera histórica se ha definido el riesgo crediticio como la probabilidad de que un prestatario caiga en *default*. Por lo tanto, un modelo de riesgo crediticio suele tener como objetivo estimar, predecir o pronosticar el riesgo de *default*. Este tema se abordará con mayor detalle en el caso de estudio.

A pesar de que el crédito ha evolucionado bastante y hoy en día es uno de los productos financieros más comunes, sigue teniendo retos importantes que se traducen en barreras para una adopción masiva. En específico vamos a abordar dos retos, el acceso a información crediticia del consumidor y la manera de medir el riesgo o capacidad de pago. La llegada de las fintech, empresas que usan la tecnología para ofrecer servicios financieros, parece estar cambiando el panorama. Muchas de estas compañías se están apalancando de la tecnología, en específico, procesos digitales y automatización de decisiones basadas en aprendizaje de máquina e inteligencia artificial para ofrecer sus productos o servicios [15]. Esto viene impulsado por el desarrollo y democratización de métodos y modelos matemáticos-estadísticos, además de la evolución exponencial de herramientas computacionales. Esto ha revolucionado y sofisticado la manera de recopilar información, procesarla y ampliar los caminos para cuantificar el riesgo y tomar mejores decisiones. Es importante entender el peso que tienen este par de retos. El no contar con información suficiente y métodos de medición adecuados eleva el riesgo de ofrecer el servicio, lo cual se traduce en condiciones desfavorables para el consumidor, que bajo el contexto crediticio significa tasas de interés demasiado elevadas. Por lo tanto, el avance en estos temas permitirá la creación de productos financieros más justos y competitivos, además de la inclusión de más personas al sistema financiero. Se puede consultar el contenido de [31] para profundizar en esta relación.

Dado el panorama anterior, la motivación de esta tesis es utilizar las matemáticas, la estadística y la computación para desarrollar un modelo de riesgo crediticio para una fintech mexicana. Esta empresa le presta dinero a personas con deuda actual en sus tarjetas de crédito, para que

liquiden dicha deuda y la transfieran con una menor tasa de interés. La metodología utilizada en el desarrollo del trabajo fue la siguiente:

1. Entender el caso de estudio;
2. Analizar los datos disponibles;
3. Definir el objetivo y lineamientos de la solución;
4. Creación de un modelo de aprendizaje de máquina;
5. Evaluación e implementación del modelo.

Esta metodología es el instructivo general que se siguió para la consecución del objetivo. Uno de los **lineamiento** generales del trabajo fue construir una solución que se adapte a las necesidades del negocio, brinde mejoras medibles y que sea escalable e iterable, de ahí la importancia del paso uno, dos, tres y cinco. Para efectos de este trabajo, revisaremos más a detalle el proceso de creación del modelo, el cual está compuesto de los siguientes pasos:

1. Recopilación **y construcción** de variables explicativas;
2. Diseño y construcción de la variable respuesta;
3. Análisis exploratorio de datos;
4. Diseño de la base de datos final para el entrenamiento del modelo;
5. Preprocesamiento de los datos;
6. Selección de modelos de aprendizaje supervisado;
7. Entrenamiento y validación de los modelos;
8. Evaluación de resultados;

9. Diseño y construcción del modelo de toma de decisión.

Ambas metodologías son una mezcla entre mejores prácticas, casos de éxito utilizados como referencia y aprendizajes propios a partir de iteraciones anteriores. Este último proceso se abordará con mucho mayor detalle en capítulos posteriores.

El escrito está organizado en cuatro capítulos posteriores. El próximo cubre la revisión del caso de estudio, para entender el problema, los datos disponibles y posibles soluciones. En el siguiente se describen las técnicas, modelos estadísticos-computacionales y métricas de evaluación a utilizar, siguiendo el mismo proceso descrito en el paso dos de la metodología general. Después continuamos con la presentación de resultados y discusión de los mismos. Por último, presentamos las conclusiones generales y consideraciones sobre próximos pasos para trabajos a futuro.

Capítulo 2

Caso de estudio

En este capítulo haremos una breve introducción a la empresa, tipo de servicio, tipo de cliente y en qué consiste el reto de construir un modelo de riesgo crediticio. También se habla sobre la importancia de los datos para efectos de los modelos, la descripción general de la construcción de las bases de datos y se presentan algunos resultados del análisis exploratorio. Una vez que tenemos buenos datos, explorados y organizados, podemos pensar en la construcción de un modelo.

La empresa de estudio es una fintech mexicana que nace con la misión de crear productos financieros justos, transparentes y que ayuden a las personas a alcanzar el bienestar financiero. En particular, se dedica a otorgar créditos en línea, es decir, por medio una aplicación desde su página de internet. Su producto estrella consiste en refinanciar deuda de tarjetas de crédito, ofreciendo una tasa de interés menor a la actual, creando un acuerdo ganar-ganar. Su modelo consiste en captar personas que tienen x deuda en su tarjeta de crédito

con una tasa de interés α , evaluar su perfil para decidir si prestarles $y \leq x$ para que aporten o liquiden su deuda de la tarjeta y obtengan una nueva tasa de interés $\beta < \alpha$.

El presente trabajo se realiza para efectos de este producto y de los clientes correspondientes. Al ser un negocio 100 % digital, gran parte de la interacción con los clientes ocurre por medio de la página. Uno de los principales objetivos del negocio es lograr identificar quiénes de los clientes que aplican por un préstamo pagarán de vuelta, pues para que el negocio sea rentable tiene que colocar dinero que genera un retorno con un interés. Entonces, un proceso clave es recopilar la información necesaria del cliente que solicita el crédito. Para efectos de este trabajo no analizaremos el diseño de la solicitud de crédito, sino que tomaremos como base la información que actualmente se le pide a un cliente. El objetivo de este trabajo es diseñar un modelo que pueda utilizar dicha información para pronosticar que tan buen o mal pagador será y entonces tomar una decisión sobre si aprobar o rechazar la solicitud. La consecución de este objetivo se medirá en función de los resultados de las decisiones que se hubieran tomando, es decir, si a los clientes que se les prestó **si** pagaron de vuelta y a los clientes que no se les aprobó el crédito en efecto eran malos clientes.

Es importante definir un punto de comparación para evaluar la solución propuesta. En un escenario ideal buscaríamos decisiones 100 % efectivas, pero esto no ocurre en la vida real. La manera natural de definir este punto de comparación es evaluando las decisiones que se han tomando hasta el momento, representándolas como el modelo de toma de decisión actual, el cuál será el modelo a vencer. Para esto, es necesario describir el proceso de captación de un cliente. La primera

interacción ocurre cuando el cliente inicia una solicitud de crédito. Después, el cliente abandona la solicitud o llega hasta el final y completa toda la información requerida. Una vez que la solicitud se completa, pasa a una revisión manual donde se toma la decisión de aprobación o rechazo. Esto quiere decir, que el conjunto de solicitudes de estudio son aquellas que ~~se~~ llegaron hasta la revisión y que el modelo a vencer son dichas decisiones manuales.

Para pensar en un modelo estadístico-computacional es necesario construir una base de datos que contenga múltiples características del cliente que aplica por el crédito. Estas características se conocen como variables descriptivas, independientes o explicativas. Entonces, el primer reto es definir esas características que describen al cliente. Una vez definidas, se recopilan o construyen para obtener una matriz en formato tabular, en la cual cada solicitud corresponde a un renglón y cada característica a una columna. En nuestro caso, contamos con las siguientes fuentes de información:

- La solicitud de crédito, donde obtenemos información personal del cliente como edad, estudios, ocupación, estado civil, dependientes económicos, ingresos, información de las tarjetas a refinanciar, etc.
- El reporte de Buró de Crédito, una Sociedad de Información Crediticia que recopila las interacciones de las personas con instituciones reguladas que otorgan créditos [2]. Del reporte obtenemos el historial crediticio del cliente, específicamente tenemos variables como la fecha de ingreso a buró, cuentas anteriores con atraso, número de consultas del reporte, número de créditos activos, deuda total actual, etc.
- Estados de cuenta más recientes, asociados a la(s) tarjeta(s) a

refinanciar. El estado de cuenta es un reporte mensual emitido por la institución que otorga el crédito describiendo el estado de la cuenta, en términos de movimientos y deuda actual. A partir de estos obtenemos métricas como la deuda total, la tasa de interés, el promedio de compras, el promedio de pagos, etc.

- Utilizando las interacciones anteriores del mismo cliente con la empresa, construimos dos grupos de variables, para los clientes que aplican.
 - A partir de sus solicitudes anteriores construimos variables como número de solicitudes anteriores, promedio de monto solicitado, desviación estándar del ingreso reportado, etc.
 - A partir de sus créditos anteriores construimos variables como número de créditos anteriores, créditos activos, máximo de días de atraso en sus pagos, promedio de monto mensual de sus pagos, etc.
- Los datos sobre la conexión del usuario, de donde obtenemos el nombre del sistema operativo del dispositivo desde donde se realizó la conexión y el nombre del buscador que se utilizó.

Se consultó de manera histórica todas las solicitudes que llegaron al proceso de análisis recibidas desde el 08/01/2019 hasta el 31/11/2021, logrando recopilar 8,811 observaciones, siendo esta nuestra ventana de observación. A partir de estas fuentes de información se logró extraer y construir 146 variables descriptivas para cada solicitud de crédito. Se puede consultar el anexo A.1.1 para conocer más detalle de las variables. Para la incorporación de información sobre interacciones previas del cliente, se tomó una “fotografía” de la información que se conocía hasta el momento que llega la solicitud en cuestión. Esto

genera un problema, ya que estas variables sólo tendrán información para clientes recurrentes, o mejor dicho, todas las primeras solicitudes de los clientes no tendrán información de interacciones anteriores. Esta discusión sale del alcance de este trabajo, pero se abordó como una consideración importante al momento de discutir resultados con la empresa, para revisar el efecto sobre clientes nuevos y recurrentes.

Una vez que tenemos las características de cada solicitud es necesario definir quién es un cliente “riesgoso”, “malo” o simplemente a quién queremos prestarle dinero. Como se mencionó en la introducción, de manera histórica el concepto clave ha sido el *default*, entendido como el hecho de que un prestatario incumpla en sus obligaciones de pago de un crédito. Para profundizar en este concepto es importante definir que significa incumplir. Supongamos que la empresa le presta a un cliente y se define que por los próximos doce meses cada día quince de mes se hará un pago. Por lo tanto, si el cliente no paga su mensualidad el día quince (o antes) estaría incumpliendo, cayendo en un atraso en sus pagos. Es importante mencionar que no es lo mismo si el cliente se atrasó un día, un mes o simplemente dejó de pagar el resto de la deuda. Y tampoco es lo mismo si un cliente se atrasó en su primera, sexta o última mensualidad. De aquí surgen las primeras consideraciones importantes para entender el *default*, a partir de qué momento se considera que un cliente cayó en incumplimiento, cómo distinguir la magnitud de dicho incumplimiento y definir la duración de la ventana de observación sobre la cuál se medirá el incumplimiento. El estándar suele ser considerar que un cliente cayó en *default* cuando su atraso es mayor o igual a treinta días durante los primeros doce meses del crédito. Sin embargo, existe una metodología para definir dichos parámetros que se

llama “análisis de cosechas” (*cohort* o *vintage analysis*). Esta metodología sale del alcance de este trabajo, pero se puede consultar la referencia [4] para obtener más detalles. Después de realizar dicho análisis, validamos que la selección de treinta días parece estar bien justificada. Por otro lado, decidimos que la ventana de observación simplemente será la duración original del crédito y usaremos esta duración como una variable descriptiva de cada solicitud. Por lo tanto, para nuestro caso de estudio la variable de default toma el valor de 1 si el cliente se atrasó treinta días o más en cualquiera de sus pagos durante la duración del crédito y 0 en cualquier otro caso. Adicional a esto se confirmó con la empresa que esta definición de incumplimiento se podría trasladar a pensar en un cliente “bueno”, “malo” o en aquellos clientes a los que si o no se les quiere prestar dinero. De esta manera, generamos un problema de clasificación binaria, donde vamos a etiquetar cada solicitud de crédito con unos y ceros. Así obtenemos nuestra variable objetivo o variable respuesta, la cuál separa a la población en dos clases y es la misma que guiará el proceso de aprendizaje del modelo. Es decir, buscaremos construir un modelo que utilice las características de cada solicitud y la etiqueta de *default* para aprender la separación entre las clases y podamos pronosticar la medida de pertenencia o probabilidad de que una nueva solicitud pertenezca a alguna de las clases. Se puede consultar la sección 3.5 del libro [17] o la sección 1.2 del libro [18] para profundizar en el concepto de *default*, este autor es uno de los mayores expertos en temas de medición del riesgo crediticio.

Es importante mencionar que para lograr esta clasificación o etiquetado de cada solicitud es necesario que ocurran las siguientes cosas. En primer lugar, que la solicitud se haya convertido en un

crédito. Es decir, que el cliente haya completado su solicitud, la empresa haya aprobado la solicitud y se haya firmado el contrato para iniciar el crédito. En segundo lugar, para poder clasificar dicha solicitud como *default* o 1 es necesario que el cliente haya caído en algún atraso de al menos treinta días y para clasificar como *no default* o 0 **es necesario que el cliente haya terminado de pagar su crédito** y nunca haya caído en un atraso mayor a veintinueve días. Esto tiene una serie de implicaciones muy importantes con respecto a los datos.

El primer hecho nos limita a conocer la etiqueta de todas las solicitudes que se rechazaron o no se convirtieron en créditos. Por lo tanto, el modelo sólo podrá observar solicitudes que se convirtieron en créditos. Esto genera una pérdida de información y un sesgo implícito, pues sólo se estará observando el tipo de cliente que se decidió aprobar y esto puede ocasionar problemas de generalización. Este es un obstáculo importante en nuestro contexto, sin embargo su discusión y posibles soluciones para mitigar el impacto quedan fuera del alcance de este trabajo. Se puede consultar el artículo [1] para profundizar en este problema.

 El segundo hecho nos impide clasificar clientes que aún no han caído en un incumplimiento mayor o igual a treinta días pero que no han terminado de pagar su crédito. Esto aumenta el problema de pérdida de información.

Como el modelo sólo utiliza observaciones etiquetadas, nuestro conjunto de datos final se conforma sólo por observaciones etiquetadas. Hasta este momento del total de 8,811 solicitudes que se recopilaron, sólo se lograron etiquetar 2,354. Para crecer el número de



observaciones, realizamos un análisis de qué clientes se podrían considerar como buenos a pesar de no haber liquidado su deuda. Encontramos que aquellos clientes que ya tienen el 90 % de su crédito pagado y no han caído en incumplimientos o atrasos en sus pagos terminando siendo buenos clientes. Por lo tanto, incluimos este criterio al esquema de clasificación y logramos etiquetar 80 clientes más.

De manera resumida, logramos construir un conjunto de datos con 2,434 observaciones, 146 variables descriptivas y una variable objetivo binaria. Sobre este conjunto y estas variables se hará todo el trabajo posterior. Es importante mencionar que para mantener la confidencialidad de la información de la empresa, los datos sensibles se presentan con una transformación que mantiene distancias pero modifica la magnitud. En particular, los niveles de *default* sufrieron esta transformación, pero los niveles se mantienen consistentes a lo largo del trabajo.

Capítulo 3

Técnicas y Modelos

En este capítulo se plantean las técnicas matemáticas, estadísticas y computacionales, así como los modelos de aprendizaje que se utilizan. Abordaremos cada elemento siguiendo el mismo orden en el que aparecen dentro del proceso de construcción del modelo. En específico se presentan los siguientes temas:

1. Separación del conjunto de datos;
2. Técnicas de preprocesamiento;
3. Modelos de clasificación;
4. Métricas de evaluación;
5. Entrenamiento y validación del modelo.

Se hará una descripción general de cada elemento, haciendo mención de algunos conceptos teóricos fundamentales y presentando una serie de ejemplos prácticos.

3.1. Separación del conjunto de datos

Recordemos que estamos construyendo modelos de aprendizaje supervisado para un problema de clasificación binaria. Por lo tanto, el aprendizaje viene a partir de observar un conjunto previamente clasificado para la estimación de parámetros o deducir las reglas de clasificación. Por otro lado, uno de los principales objetivos de un modelo es su capacidad para generalizar sobre nuevas observaciones, que se puede entender como la capacidad de clasificar una nueva observación de manera correcta. Esto está ligado al concepto de sobre-ajuste, que se dice cuando la estructura del modelo se definió para un conjunto de observaciones muy particular y no representa a la población completa. Para mitigar este riesgo y aumentar la capacidad de generalización del modelo, existen algunos conceptos y técnicas muy importantes a seguir en el proceso de construcción. El concepto principal es el de separar la muestra en dos subconjuntos de observaciones que serán utilizados para distintos objetivos: entrenar, validar y probar; de ahí los nombres de cada uno de los conjuntos. El conjunto de entrenamiento sirve para entrenar y definir la estructura del modelo. Por lo tanto, este conjunto debería ser el más grande para poder adquirir el mayor aprendizaje posible. El conjunto de validación sirve para validar el modelo y poder ajustar hiper-parámetros a partir de ciertas métricas. El conjunto de prueba, es un conjunto que no se utiliza para construir el modelo, sino que se usa para medir la capacidad predictiva y el poder de generalización. Esta definición de los conjuntos presenta algunas limitantes. La principal, que para validar necesitamos un conjunto “grande”, pero es algo que no se suele tener al momento de querer construir un modelo. Esto llevó a buscar cómo unificar los conjuntos de entrenamiento y validación, para

utilizar más datos para entrenar el modelo dada una estructura y encontrar la estructura óptima. El desarrollo de algunas técnicas de simulación, en particular la técnica de validación cruzada (misma que se describe más adelante), permite cumplir ambos objetivos utilizando el mismo conjunto. Por lo tanto, ahora la separación sólo consistirá en un conjunto de entrenamiento y otro de prueba. De manera histórica se ha mantenido la regla de que el conjunto de entrenamiento sea “mayor” que el de prueba, pero no hay reglas preestablecidas para determinar la proporción exacta. Puede ser 60-40, 70-30, 80-20, o algo por el estilo. Para nuestro caso, dado que no tenemos muchas observaciones, utilizaremos la proporción 70-30. Es decir, 70 % de las observaciones se utilizarán como conjunto de entrenamiento y 30 % como conjunto de prueba. Esta partición se realiza por medio de un muestreo aleatorio sin reemplazo. Si tenemos 100 observaciones, se eligen 70 para entrenar y 30 para probar de manera aleatoria. En la práctica, suele agregarse la condición de “estratificar” los conjuntos con respecto a alguna variable categórica importante. Esto consiste en hacer la partición tal que cada parte contenga la misma proporción de cada una de las clases de dicha variable. Lo más común dentro de problemas de clasificación es que la estratificación se realice a partir de la variable respuesta, en nuestro caso, la clasificación de clientes “buenos” y “malos”. Esto con la intención de homologar los conjuntos al menos en proporción de clases, evitando un sesgo a priori y logrando generalizar mejor.

Para nuestro caso de estudio logramos construir un conjunto de datos con 2,434 observaciones, 146 variables descriptivas y una variable binaria que nos dice si un cliente fue “bueno” o “malo”. Para mostrar la separación de los conjuntos y el efecto de la estratificación

con respecto a la variable objetivo, mostramos una tabla con la distribución final de los conjuntos.

Cuadro 3.1. Distribución final de los conjuntos de datos

Conjunto	Observaciones	% del total	% de malos
Completo	2,434	100 %	22.55 %
Entrenamiento	1,703	69.96 %	22.54 %
Prueba	731	30.04 %	22.57 %

El principal objetivo de hacer la separación es simular que sólo se tiene el conjunto de entrenamiento para construir y luego se tiene un conjunto independiente para validar. Por lo tanto, usaremos el conjunto de entrenamiento para definir el preprocesamiento, entrenar y ajustar los modelos; logrando mitigar el sobre-ajuste y evitando incluir un sesgo al conjunto de prueba. Entonces, el conjunto de prueba se mantendrá intacto e invisible, hasta que tengamos el proceso completo para hacer nuevas predicciones y entonces lo alimentemos con este conjunto y obtengamos nuestros resultados de prueba.

3.2. Técnicas de preprocesamiento

Previo a la construcción de un modelo es necesario tomar algunas decisiones con respecto al tratamiento de los datos. En la práctica es casi inevitable encontrar datos faltantes, enfrentarte con variables de distinta naturaleza y estructura, tener distintas proporciones entre clases, observar variables con distribuciones “extrañas”, tener observaciones atípicas, entre muchos otros problemas. **El proceso de preprocesamiento** es el conjunto de tratamientos que nos permite preparar los datos para poder alimentar un modelo. **Este proceso**

contiene los pasos:

1. Tratamiento de datos faltantes;
2. Técnicas de codificación para variables no numéricas;
3. Transformación de variables;
4. Selección de variables;
5. Tratamiento para datos no balanceados.

Estas técnicas y tratamientos serán el objetivo de esta sección.

3.2.1. Tratamiento de datos faltantes

En la vida real, es casi imposible tener todos los datos completos, ya sea por omisión, por errores en el registro de la información o alguna otra causa. Ante esta situación y debido a que la gran mayoría de modelos no funcionan si tenemos datos faltantes, es necesario definir distintas políticas para solucionar el problema. La política o regla a aplicar depende del tipo de variable, sea numérica o categórica, y de las circunstancias o limitantes del problema. Podríamos pensar que lo más sencillo y correcto sería deshacernos de las observaciones que contienen datos faltantes, para evitar incorporar algún tipo de sesgo. Pero esto suele ser muy costoso en términos de perder un gran número de observaciones que puedan ayudar a obtener mejores resultados. Entonces, si no queremos perder dichas observaciones, tenemos que recurrir a imputar o reemplazar los datos faltantes. Para el caso de variables numéricas existen un par de opciones comunes. La primera opción es reemplazar los datos faltantes con ceros si el cero “hace sentido”. Por ejemplo, para variables que contabilizan eventos,

podríamos decir que el cero significa que no se observó ningún evento. La segunda opción es reemplazar con alguna métrica de agrupación (media, mediana o moda), bajo el supuesto de que si no observamos un valor y lo tomamos con la media, no estaríamos incorporando sesgo negativo ni positivo sino sería “uno más del promedio”. Para el caso de variables categóricas existen **otras dos opciones** comunes. La primera aplica cuando el número de observaciones con datos faltantes es muy pequeño. En este caso podemos utilizar la moda, es decir reemplazamos con la categoría más frecuente. La segunda opción, cuando tenemos un mayor número de observaciones con datos faltantes, es reemplazar con una nueva categoría del tipo “indefinido” o “*Unknown*” para representar a todas esas observaciones como una nueva clase.

Vamos a revisar nuestros datos y experimentar estas técnicas. Iniciamos por observar para que variables tenemos datos faltantes y cuántas observaciones son.

Cuadro 3.2. Ejemplos de variables con datos faltantes

Variable	Datos Faltantes
<i>browser_name</i>	515
<i>housing_type</i>	6
<i>maxDaysLate</i>	185
<i>meanIncome</i>	116

En general, todas las variables con datos faltantes mantienen la representación en el conjunto de entrenamiento. Esto es de gran utilidad, pues utilizando únicamente el conjunto de entrenamiento, lograremos definir las reglas necesarias para el tratamiento en todo el

conjunto. Si quisiéramos eliminar las observaciones con datos faltantes de todo el conjunto, es decir, eliminar cualquier observación que al menos en una de sus variables tenga un dato faltante, nos quedaríamos con 1,431 observaciones y perderíamos 976 observaciones equivalentes al 40.54 % de nuestros datos. Esto es clara evidencia del tipo de situaciones en la vida real, eliminar los datos se vuelve una opción demasiado costosa e inviable. Con excepción de *housing_type* y *marital_status* todas las otras variables con datos faltantes tienen más de 100 registros que podrían considerarse “muchos” con respecto al total de observaciones (más del 4 %). Por lo tanto para estos dos casos, utilizaremos la técnica de reemplazo con el valor más frecuente, pues ambas son variables categóricas. Para *maxDaysLate* utilizaremos el reemplazo con el cero, pues estamos contabilizando el máximo de días de atraso y cero días es un valor válido. Para las variables numéricas continuas utilizaremos el reemplazo con la media. Para el caso de las variables categóricas el reemplazo con una nueva clase llamada “Unknown”. De manera conjunta, el tratamiento queda de la siguiente manera.

Cuadro 3.3. Ejemplos de los distintos tratamientos para datos faltantes

Variable	Tratamiento
<i>browser_name</i>	Nueva clase
<i>housing_type</i>	Clase más frecuente
<i>maxDaysLate</i>	Reemplazo con cero
<i>meanIncome</i>	Reemplazo con la media

Esta tabla nos dice explícitamente una regla de asociación. Esta tabla se guarda y se utiliza para el tratamiento de nuevos datos, tanto

para el conjunto de prueba como para cualquier observación nueva. Cabe recalcar que todos los valores a utilizar para reemplazar los datos faltantes se calcularon sobre el conjunto de entrenamiento.

3.2.2. Técnicas de codificación para variables no numéricas

La gran mayoría de los modelos estadísticos computacionales reciben datos numéricos y regresan un resultado numérico. Es por ello, que ante la presencia de datos categóricos o variables con texto, se suele usar alguna técnica para codificarlos y convertirlos en datos numéricos. En esta ocasión, pondremos a prueba tres de las técnicas más utilizadas en la práctica, la creación de variables *dummies* o *One Hot Encoding*, la codificación ordinal y la combinación de categorías junto con alguna de las dos anteriores.

La técnica de creación de variables *dummies* consiste en codificar una variable categórica en un arreglo de variables binarias. Para cada categoría se crea una variable con valor de uno si la observación pertenece a esa categoría y cero en otro caso. Por complemento, no es necesario construir n variables para n categorías, con crear $n - 1$ variables binarias estaríamos representando toda la información. Para ejemplificar la técnica, vamos a utilizar la variable *user_occupancy_type*, la cual tiene las categorías de “Empleado”, “Negocio propio” o “Retirado”. Utilizando este método estaríamos creando una variable que se llama *user_occupancy_Empleado* que contenga uno para los clientes que son empleados y cero en cualquier otro caso. Otra variable *user_occupancy_Negocio* equivalente. Esta técnica se utiliza cuando la variable categórica es nominal y no tiene orden alguno. Si la(s)

variables(s) a codificar tienen muchas clases este método podría traer problemas porque estaríamos creando demasiadas nuevas variables.

Por otro lado, la técnica de codificación ordinal consiste en transformar una variable categórica en una variable numérica con enteros que van del cero hasta el número de categorías menos uno. Es decir, a cada categoría se le asigna un entero y se hace el mapeo correspondiente, comenzando por el cero, uno, dos, hasta $n - 1$. Este método se utiliza para variables categóricas ordinales, donde ya existe un orden entre clases y mantenerlo es importante. Vamos a utilizar la variable *user_level_of_studies* que tiene los siguientes valores: “Secundaria”, “Preparatoria”, “Licenciatura” y “Posgrado”. Aplicando el método pasaríamos a tener cero para los que llegaron hasta la secundaria, uno para los que llegaron a preparatoria, dos para los de licenciatura y 3 para los de posgrado. Notemos que para el caso de variables binarias aplicar la codificación ordinal es equivalente a aplicar una codificación binaria.

Por último, la técnica de combinación de categorías consiste en agrupar categorías “raras” o poco representativas mediante alguna regla. Este método sirve para reducir la dimensionalidad de variables con “muchas” clases. Cabe recalcar que una vez que aplicamos la combinación de categorías seguimos teniendo una variable categórica. Por lo tanto, aún hay que aplicar alguno de los primeros dos métodos según sea el caso. La regla de agrupación puede venir de cierta intuición o lógica de negocio, por frecuencias o por niveles de ocurrencia con respecto a la variable respuesta. Para combinar por frecuencias, primero vemos la frecuencia de cada una de las categorías y tomamos un porcentaje de corte del tipo: las clases con menos de

5 % de frecuencia se agruparán. Por niveles de ocurrencia con respecto a la variable respuesta, primero asignamos el porcentaje de *default* de cada clase y podemos agrupar a todas las clases con > 0.9 , entre $[0.9, 0.8)$ y así sucesivamente.

A partir de las tres técnicas descritas, dividiremos las variables en tres grupos de tratamiento:

1. Para variables categóricas ordinales o binarias usaremos la codificación ordinal.
2. Para variables categóricas nominales con pocas categorías usaremos la creación de variables *dummies*.
3. Para variables categóricas con muchas categorías usaremos la combinación de categorías más alguna de las codificaciones.

Para consultar más sobre el tratamiento de variables categóricas se puede consultar las referencias [8] y [28].

Como vamos a aplicar distintos métodos de acuerdo a la naturaleza de cada variable, el primer paso debe ser clasificar todas nuestras variables. La clasificación consiste en variables continuas, discretas, nominales, ordinales y binarias. Pero para tener una distinción más precisa y ahorrar esfuerzos proponemos la siguiente clasificación más granular: continuas, discretas, nominales, numéricas ordinales, categóricas ordinales, numéricas binarias y categóricas binarias. De acuerdo a los tres tipos de tratamiento que se describieron y la propuesta de clasificación, concluimos con el siguiente tratamiento.

Tipo de variable	Método	Ejemplo de variables
Nominales (pocas clases)	Variables <i>dummies</i>	<i>housing_type</i>
Nominales (muchas clases)	Combinación + codificación	<i>user_state</i>
Númericas ordinales	Codificación ordinal	<i>economic_dependents</i>
Catégoricas ordinales	Codificación ordinal	<i>user_level_of_studies</i>
Catégoricas binarias	Codificación ordinal	<i>user_gender</i>

Cuadro 3.4. Tipo de variables y codificación a utilizar

Primero, observamos que las variables continuas, discretas y numéricas binarias, ya no tienen que sufrir ninguna codificación. Para el caso de variables nominales, definiremos pocas clases como cinco o menos y muchas clases como más de cinco. Esto podría parecer muy sesgado, pero son el tipo de decisiones sencillas que podemos obviar y seguir adelante. Por lo tanto, para las variables nominales con más de cinco categorías, primero observaremos cómo combinar y luego cómo codificar. Para las variables numéricas ordinales, se menciona una simplificación en la codificación, pues para el caso de que alguna variable ya tenga el mismo formato de salida del método (número enteros continuos del 0 al $n - 1$) nos ahorraremos la codificación. Para el resto de los casos, se aplicará el método descrito anteriormente.

Antes de comenzar a ejemplificar las codificaciones vamos mencionar una consideración importante. Al trabajar sobre el conjunto de entrenamiento nos podemos enfrentar ante el problema de no observar una clase que sí aparece en el conjunto completo. Para resolver este problema podemos abordar tres caminos distintos. El primero consiste en hacer la codificación sobre el conjunto de datos completo. El segundo consiste en asignar una regla para codificar las clases no observadas. El tercero consiste en hacer la división estratificada tal que cada clase esté representada en ambos conjuntos.

En el primer caso, la pregunta es cuál es el riesgo de codificar antes de separar los conjuntos comparado contra el posible efecto de no tener representación de alguna clase. Esta es una pregunta muy complicada, pues vista la teoría sabemos que como el conjunto de prueba en principio no lo conocemos no lo podemos utilizar para ningún tipo de tratamiento al momento de construir el modelo. Pero, por otro lado, no tenemos una medida clara del impacto que puede tener ignorar alguna clase que a pesar de que no será utilizada para entrenar sabemos que existe. En el segundo caso, lo más sencillo sería crear un clase extra llamada “Unknown”, en la cuál caigan todas las clases no observadas y ya a partir de ahí aplicar el mapeo correspondiente. En el tercer caso tenemos algunos problemas de jerarquía, pues como mencionamos anteriormente, la estratificación más importante es en torno a la variable respuesta. Si quisiéramos seguir estratificando con respecto a otras variables, posiblemente estaríamos sesgando demás las muestras o simplemente llegaríamos a un problema sin solución por la intersección nula de los conjuntos. Siguiendo nuestra intuición teórica, procederemos a tomar el segundo camino en caso de tener problemas al momento de la codificación. Este tema se dejará como nota en los pasos a futuro, pensando en que hace falta más investigación y pruebas para decidir cuál camino es el correcto de acuerdo a distintos contextos.

Recordando los posibles caminos para combinar categorías, vamos a comenzar a ejemplificar la creación de una regla de asociación por intuición o lógica de negocio utilizando la variable de *user.state*. Existe una agrupación de estados en cinco categorías de acuerdo a su posición geográfica (noroeste, noreste, occidente, centro y sureste), entonces haría sentido utilizar dicha regla que nos permite pasar de 32 estados a 5

zonas. Es importante analizar los efectos de la agrupación en términos de frecuencia y tasa de *default*. La primera, para validar si la agrupación logró clases representativas. Sería deseable que todas las clases tengan al menos un 5 % de observaciones. La segunda, para validar que cada clase aporte información distinta con respecto a la variable respuesta. Sería deseable que cada clase tenga distintas tasas de *default*, porque sino más de una clase nos estaría aportando la misma información de manera individual. Recordemos que por temas de privacidad de los datos, no mostraremos los nombres explícitos de las clases ni las tasas reales de *default*.

Clases	Frecuencia (%)	Tasa de <i>default</i>
Clase 1	35.70	25.16
Clase 2	10.51	19.55
Clase 3	9.57	23.31
Clase 4	33.58	17.65
Clase 5	10.62	31.14

Cuadro 3.5. Nueva agrupación para *user_state*

Podemos ver que dicha agrupación fue muy buena, pues cada clase tiene más de 9 % de observaciones y todas tienen distintas tasas de *default*. Por lo tanto nos quedaremos con esta combinación y ahora aplicaremos la creación de variables *dummies* ya que las clases no tienen ningún orden en específico.

Ahora para ejemplificar una regla utilizando la frecuencia de cada clase usaremos la variable *bank*. Observamos en los datos que hay tres bancos con más del 10 % de observaciones, cuatro bancos entre el 5 – 10 % y el resto con menos de 5 %. Para clases con alta frecuencia,

hace sentido no combinarlas pues tenemos suficientes datos para evaluarlas por si solas. Entonces dejaremos los tres bancos con más de 10% por separado. Para el resto, podemos proponer la clase de “Bancos medianos” para agrupar los cuatro bancos entre el 5 – 10% y la clase de “Bancos pequeños” para agrupar el resto de los bancos.

Por último, ejemplificaremos una tercera regla de asociación con la variable *credit_card_name*. Una nota importante es que hicimos un ajuste a esta variable concatenando el nombre del banco antes del nombre de la tarjeta. Logrando una distinción más precisa entre tarjetas, obteniendo así 126 clases. Primero, intentamos agrupar utilizando una regla intuitiva (por el conocimiento de la industria). La propuesta consistía en agrupar tarjetas de acuerdo a las siguientes clases: clásicas, oro, platino y tarjetas emitidas por bancos digitales o *fintechs*. El problema aquí fue que las tarjetas clásicas representan más del 60% de la población y las tarjetas emitidas por *fintechs* sólo aportan el 2%. Además, hay un gran número de tarjetas que se han visto muy pocas veces (menos de diez observaciones) y nos gustaría poderlas tratar por separado. Dado que no sabemos si su comportamiento es real o está sesgado por la información tan limitada. Esto nos llevó a plantear una agrupación más adecuada al problema, utilizando tanto las frecuencias como las tasas de *default*.

1. *new cards*: tarjetas con menos de 10 observaciones;
2. *low risk cards*: tarjetas con una tasa de *default* $\leq x$;
3. *medium risk Cards*: tarjetas con una tasa de *default* $\in (x, \text{media}]$;
4. *high risk cards*: tarjetas con una tasa de *default* $\in (\text{media}, y]$;
5. *very high risk cards*: tarjetas con una tasa de *default* $> y$.

Es importante mencionar que estos “niveles de riesgo” se discutieron con la empresa para que respondieran a su aversión al riesgo, por lo que también se mantienen confidenciales. Con esta agrupación logramos reducir 126 clases a tan sólo 5, manteniendo una buena distribución de frecuencias y una clara distinción en los niveles de riesgo.

Siguiendo este tipo de lógicas logramos combinar todas las variables nominales con más de cinco clases, obteniendo a lo más cinco categorías. Para todas estas variables aplicaremos la creación de variables *dummies*, pues ninguna de ellas tiene algún orden lógico y sería incorrecto aplicar una simple codificación ordinal. Para el resto de características nominales aplicaremos la creación de variables *dummies*. Para todas las demás aplicaremos la codificación ordinal, haciendo un ajuste manual para mantener el orden lógico de las características ordinales. Es decir, si ya conocemos el orden de una variable la codificación se definirá manualmente para mantener dicho orden.

3.2.3. Transformación de variables

Es importante recordar que los supuestos bajo los cuales se construye un modelo o algoritmo se vuelven decisivos al momento de voltear a ver los resultados y poder confiar en ellos. Utilizar un modelo sin cumplir los supuestos más importantes sería equivalente a ir al dentista por un dolor de estómago y esperar a que el doctor resuelva tu problema. El diagnóstico del dentista podría tener algo de certeza, pero no sabríamos si por mera intuición o por algo más. De manera puntual, uno de los principales supuestos en todo el mundo estadístico (no bayesiano) es el supuesto de normalidad, pero allá afuera nos

encontramos con muestras de variables con todo tipo de distribuciones. Es por esto, que si estamos trabajando bajo normalidad y una de nuestras muestras no parece cumplir el supuesto, será necesario aplicar alguna transformación para “normalizar” la variable. Indagando un poco más en las causas de estos problemas, solemos encontrarnos observaciones atípicas o *outliers*, observaciones que se comportan muy distinto del resto. En la práctica, los *outliers* pueden ser ocasionados por errores en la información o por que en realidad se presentó algo fuera de lo común. En nuestro caso identificamos un gran número de variables con problemas de *outliers* y dividiremos el tratamiento según la causa del problema. Para los casos generados por errores en la información se intentará plantear un rango o soporte que **haga sentido** y se harán los ajustes correspondientes. Por ejemplo, sabemos que la variable *user_age* no puede ser menor a 18, pues un menor de edad no puede solicitar un crédito. Entonces, si observamos un valor menor, simplemente lo convertiremos en 18. Para el resto de los casos o las variables en donde no se logró poner un rango o soporte razonable, se buscará la mejor transformación para eliminar el problema. Se plantean las siguientes tres transformaciones como posibles opciones.

En primer lugar, la transformación logaritmo consiste en reemplazar cada valor de una variable por su logaritmo. En nuestro caso usaremos una pequeña variación denotada por $\ln(x + 1)$, para evitar errores cuando $x = 0$. Eso nos ayuda a reducir o eliminar la falta de simetría y condensar la distribución.

En segundo lugar, la **transformación Box-Cox** consiste en aplicar la transformación Box-Cox a cada valor de una variable. Esta

transformación se define de la siguiente manera.

$$x^\lambda = \begin{cases} (x^\lambda - 1)/\lambda & \text{si } \lambda \neq 0 \\ \log(x) & \text{si } \lambda = 0 \end{cases}$$

Con x un valor estrictamente positivo. Aquí se busca el valor óptimo de λ de acuerdo a cuál resulta en la mejor aproximación a una distribución normal. Esta función es parte de la familia de transformaciones potencia y es ampliamente utilizada en el contexto de series de tiempo.

En tercer lugar, la transformación Yeo-Johnson consiste en aplicar dicha transformación a cada valor de una variable. Esta familia de transformaciones se define de la siguiente manera:

$$x^\lambda = \begin{cases} ((x+1)^\lambda - 1)/\lambda & \text{si } \lambda \neq 0, x \geq 0 \\ \log(x+1) & \text{si } \lambda = 0, x \geq 0 \\ -[((-x+1)^{(2-\lambda)} - 1)]/(2-\lambda) & \text{si } \lambda \neq 2, x < 0 \\ -\log(-x+1) & \text{si } \lambda = 2, x < 0 \end{cases}$$

La gran ventaja de esta transformación es que se puede usar sin ninguna restricción sobre el valor de la variable. Esta familia preserva muchas propiedades de la transformación Box-Cox, de hecho para valores positivos es exactamente la misma. El valor de λ se busca de acuerdo al mismo objetivo que en la transformación Box-Cox.

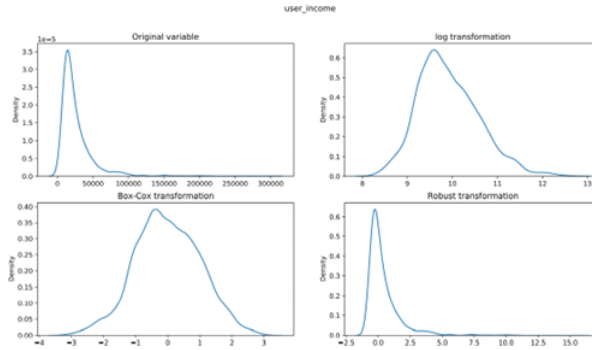
El método a seguir consistirá en aplicar todas las transformaciones posibles y construir las gráficas de distribución correspondientes. Para que de manera visual logremos identificar cuál transformación cumple mejor la función de mitigar la presencia de valores atípicos. Este proceso se seguirá para cada una de las variables numéricas que

presenten algún problema “serio” de *outliers*.

Una vez resuelto el problema de valores atípicos, se tendrá que aplicar una estandarización a todas las variables, si el modelo a utilizar tiene el supuesto de normalidad. La estandarización consiste en restar la media y dividir entre la desviación estándar a cada observación para cada una de las variables. Esto quiere decir, que existe la posibilidad de tener un conjunto estandarizado y uno no estandarizado y utilizarlos para distintos algoritmos, profundizaremos un poco más sobre esto en la explicación de cada modelo.

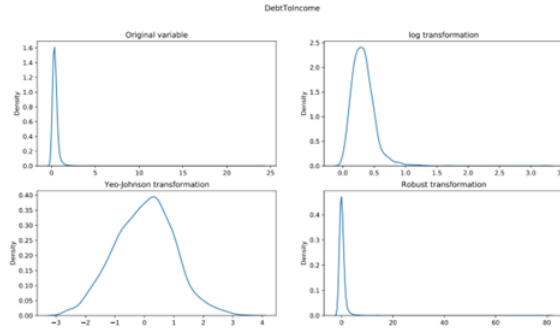
Para explorar el efecto de las transformaciones planteadas comenzaremos con un análisis univariado de la distribución de las variables continuas relevantes. El proceso para identificar variables relevantes a tratar fue analizando la situación de datos atípicos, observando máximos/mínimos o las distribuciones prestando especial atención en las colas. Una vez seleccionadas las variables pusimos a prueba las tres transformaciones descritas.

Figura 3.1. Distribución y transformaciones de la variable *user_income*



La gráfica superior-izquierda representa la distribución original, la superior-derecha el resultado de aplicar la transformación logarítmica, la inferior-izquierda el resultado de aplicar la transformación Box-Cox y la inferior-derecha el resultado de aplicar una estandarización. Podemos observar que hay una o más transformaciones que logran mitigar el efecto de valores atípicos. Sin embargo, en este caso preferimos la transformación más sencilla y/o interpretable. Entonces, nos quedaremos con la logarítmica. Recordemos que después de aplicar la transformación logarítmica aún podríamos aplicar la estandarización.

Figura 3.2. Distribución y transformaciones de la variable *DebtToIncome*



En este otro caso podemos ver dos cosas importantes. Primero, la gráfica de abajo a la izquierda corresponde a la transformación Yeo-Johnson, debido a que la variable original tenía ceros y no podíamos aplicar la transformación Box-Cox. Segundo, esta misma transformación fue la que mejor logró mitigar el problema de *outliers*.

Siguiendo esta misma lógica fuimos asignando las transformaciones a cada una de las variables problemáticas. De esta manera definimos los siguientes grupos:

1. Se aplicó la transformación logarítmica y luego estandarización a 9 variables.
2. Se aplicó la transformación Box-Cox a la variable *monthsFirstAccount*.
3. Se aplicó la transformación Yeo-Johnson a 7 variables.
4. Al resto de las variables se les aplicó una estandarización.

3.2.4. Selección de variables

La selección de variables es una parte importante al momento de construir modelos estadísticos, ya que un modelo que recibe “mala” información obtendrá “malos” resultados. ¿Qué significa tener mala información? Puede ser incluir variables que carecen de interpretación o variables constantes donde se observa el mismo valor para todas las observaciones. En términos estadísticos, mala información puede venir de incluir variables altamente correlacionadas O incluir variables que no aportan información para lograr la clasificación. Esto último se da cuando el poder predictivo o la cantidad de información que aportada una variable con respecto a la variable objetivo es muy poca o insignificante. Es por ello que se hará una selección de variables de acuerdo a las distintas situaciones descritas.

Para el caso de variables que carecen de interpretación o que son constantes, la metodología es muy directa. Si una variable presenta estas características se excluirá para efectos del modelo, ya que incluirla no ayudaría en mejorar la clasificación. Siendo más precisos en el significado de la carencia de interpretación, podríamos entenderlo como aquellas variables cuya definición o valores carecen de relación lógica con la variable a predecir. El ejemplo perfecto es el número de teléfono de un cliente, ya que todos los clientes tienen números distintos y no hay una relación directa con su comportamiento crediticio.

Para el caso de variables altamente correlacionadas, primero vamos a dar una introducción al concepto de correlación. La correlación entre dos variables se entiende como el sentido y grado de relación lineal

entre los comportamientos de dichas variables. Para medir la correlación, la herramienta estándar es el coeficiente de correlación de Pearson, denotado por $r_{X,Y}$. Decimos que dos variables están altamente correlacionadas cuando $|r_{X,Y}| \geq 0.75$. Esto es un poco arbitrario y podría cambiar de acuerdo al objeto de estudio, pero en general valores cercanos a uno serán los de mayor interés. El problema de correlación entre variables dentro de modelos lineales se conoce como multicolinealidad. Esto ocasiona que pequeñas variaciones en estas variables generan resultados muy distintos en el modelo, creando inestabilidad o sobre-ajuste. La metodología a seguir para abordar este problema consiste en hacer una matriz de correlación entre las variables explicativas, la cual contiene el coeficiente de correlación entre todos los pares de variables de forma matricial. Con esto podemos identificar las variables más correlacionadas y eliminar alguna de ellas, en caso de que además de la alta correlación tengan una relación explícita en términos de sus significados. Por ejemplo, si tenemos una variable que mide la ocurrencia de un evento en un periodo de tiempo y otra variable que mide lo mismo pero un periodo de tiempo mayor o menor. Estas variables definitivamente estarán correlacionadas y podría ser un ejemplo claro de cuándo podríamos decidir eliminar alguna de las dos.

Por último, para el caso de variables que no aportan información al problema de clasificación, es necesario establecer una medida para determinar qué se considera aportar o no aportar información y en qué magnitud. Dentro del contexto de riesgo crediticio y modelos lineales, surgen dos conceptos clave para abordar este tema. Primero, el concepto de *Weight of Evidence (WOE)*, una medida del poder predictivo de una variable independiente en relación con la variable

dependiente. Esta se puede interpretar como una medida de separación entre las clases para un grupo específico de observaciones. Segundo, el concepto de *Information Value (IV)*, una métrica que nos dice qué tanta información está aportando una variable al problema de clasificación. Consultar Apéndice A.3 para más detalles. Esto ya nos da una medida específica para medir si una variable está aportando información en términos lineales. Entonces, este criterio es útil y aplicable dentro del contexto de modelos lineales. Para abordar modelos no lineales, existen algunos métodos alternativos. Uno de ellos consiste en utilizar modelos de aprendizaje no lineal para detectar qué variables tienen un peso importante en el problema de clasificación. Es nuestro caso utilizaremos un modelo de Bosques Aleatorios, mismo que será descrito más adelante. El cual es útil debido a su capacidad intrínseca de clasificar variables de acuerdo a la importancia de sus pesos. Así como estos dos, existen muchos otros enfoques. Por lo tanto, para combinar distintos enfoques, pondremos a prueba un algoritmo reciente que combina distintos métodos de selección. La idea detrás es aplicar una variedad de métodos de selección con distintos criterios y dar un voto cada vez que cierto método selecciona una variable. Al final, se seleccionan las variables que hayan logrado una mayoría de votos. Este algoritmo se pondrá a prueba para comparar el desempeño de todos los modelos y poder concluir qué estrategia nos conviene más. Utilizaremos una implementación de dicho algoritmo llamada “Voting Scheme”, dicho algoritmo tiene 4 métodos de selección adicionales a los ya mencionados y se puede encontrar más información en la referencia [33]. Sin la intención de entrar a detalle en cada uno, pues no se les dará mucha importancia en el desarrollo del trabajo, sólo se hará el listado completo de los métodos disponibles:

1. Criterio IV usando WOE;

2. Nivel de importancia usando bosque aleatorio;
3. Eliminación recursiva;
4. Importancia usando árboles extra;
5. Criterio Chi cuadrado;
6. Criterio L1.

Para variables que presentan alguno de los problemas anteriores, se puede buscar extraer información adicional mediante la creación de nuevas variables. Esto es útil para lograr interpretación de variables que de manera directa parecen inútiles. Retomando el ejemplo del número de teléfono de un cliente, podríamos extraer algún tipo de métrica que nos diga la lada de cada teléfono, redefinir el significado de dicha variable y lograr una relación directa con la variable respuesta.

Comenzamos describiendo las variables **ha** eliminar, desde antes de entrenar cualquier modelo, agrupándolas por el motivo de eliminación.

- Falta de interpretación. El ejemplo perfecto es la variable *loan_request_id* la cual sólo nos dice la clave única de cada solicitud de crédito. Este tipo de variables suelen ser útiles para análisis exploratorios, pero se tienen que excluir al momento de entrenar un modelo. Hubo 11 variables con estas características.
- Falta de variabilidad. Un buen ejemplo es la variable *TipoMoneda* la cual tiene el valor de “MX” para todos los registros, indicando que todas las cuentas en buró estaban en pesos mexicanos. Este tipo de variable nos aportan información del conjunto pero no de cada observación. Hubo 18 variables con esta característica.

- Alta correlación. Se identificaron 8 grupos de variables con un coeficiente de correlación $\geq .85$, se seleccionó una variable “favorita” de cada grupo y se eliminaron las demás. De esta manera eliminamos 11 variables y mitigamos el problema de multicolinealidad.

Ahora sigue revisar si hay más variables que no aportan información al problema de clasificación. Para esto usaremos el método de votación, tomando cada variable y calculando la métrica o aplicando la prueba utilizada en cada uno de los métodos correspondientes.

Variable	IV	Bosq. Aleat.	RFE	Árboles	Chi Cuad.	L1
<i>openedLoans6Months</i>	0.3016	0.0169	<i>True</i>	0.0132	8.0496	<i>True</i>
<i>totalDebtCC</i>	0.0826	0.01556	<i>True</i>	0.0078	2.0605	<i>False</i>
<i>credit_card.rate</i>	0.0390	0.0172	<i>False</i>	0.0113	0.4550	<i>False</i>

Cuadro 3.6. Métricas por método de selección

A partir de cada métrica, se aplica la regla de selección de cada método y traducimos estos números en votos. Para los dos métodos que aplican pruebas, simplemente traducimos el resultado binario en un voto.

Variable	IV	Bosq. Aleat.	RFE	Árboles	Chi Cuad.	L1
<i>openedLoans6Months</i>	1	1	1	1	1	1
<i>totalDebtCC</i>	1	1	1	0	1	0
<i>credit_card.rate</i>	0	1	0	1	0	0

Cuadro 3.7. Votos por método de selección

De esta manera 56 variables recibieron tres o menos votos y se eliminaron. Una consideración importante fue no eliminar variables *dummies*, pues esto nos llevaría a tener una pérdida de información

con respecto a la variable original. Por lo tanto, esta será una técnica a probar al momento de buscar el tratamiento óptimo.

3.2.5. Tratamiento para datos no balanceados



Un problema de clasificación no balanceado es aquel que no contiene una proporción similar de observaciones para cada una de las clases o etiquetas. En nuestro contexto, lo más común es que el conjunto de datos no esté balanceado, pues la proporción de clientes que caen en *default* esperamos que sea baja. Suele existir una serie de riesgos implícitos cuando nos enfrentamos contra un problema de este tipo, entre ellos están la incorporación de sesgo accidental o la preferencia hacia alguna de las clases predominantes. Esta situación se puede atacar desde tres enfoques distintos: métodos de preprocesamiento, selección de métricas de validación y selección y configuración de algoritmos. En esta sección hablaremos de algunos métodos a nivel preprocesamiento. La técnica más sencilla es balancear el conjunto, ya sea incrementando el número de casos de la clase minoritaria (*over sampling*) o disminuyendo los casos de la clase mayoritaria (*under sampling*). Esta técnica en su versión más sencilla es un método aleatorio sin reemplazo, donde se eligen observaciones aleatorias a duplicar o eliminar según sea el caso. Sin embargo estas técnicas suelen ocasionar pérdida de información o un sobre-ajuste. De ahí surge una técnica más robusta llamada *Synthetic Minority Over-sampling Technique (SMOTE)*. Este método consiste en seleccionar un subconjunto de la clase minoritaria para crear un nuevo conjunto similar de forma sintética. Para luego agregarlo al conjunto original, mitigando el sobre-ajuste que provoca el duplicar observaciones. Se puede revisar el Apéndice A.4 para más detalles sobre esta técnica y algunas variaciones. Pondremos a prueba dicha técnica utilizando un

paquete de Python que ya contiene la implementación de este método con sus distintas variantes.

3.3. Algoritmos de clasificación

Una vez que tenemos un conjunto de datos con variables descriptivas y una variable binaria como objetivo de predicción, podemos pensar en aplicar un modelo de clasificación binaria. En particular uno de aprendizaje supervisado, pues de antemano conocemos la etiqueta de cada observación, la cuál se utilizará para “aprender”. Dado que existen distintos modelos de aprendizaje supervisado para problemas de clasificación binaria, la mejor estrategia en términos prácticos es proponer una serie de modelos, ponerlos a prueba y elegir uno o varios modelos ganadores. Recordemos que cada modelo tiene distinta estructura, supuestos y objetivos. Buscaremos cubrir de manera general estos tres aspectos en la descripción de cada uno. A nivel general podríamos pensar que buscamos un balance entre interpretabilidad y poder predictivo. Un resultado deseado sería tener un modelo sencillo, en términos de estructura, el cual tuviera un alto poder predictivo. En nuestro caso, un alto poder predictivo significa poder predecir con alta efectividad si una observación pertenece a la clase de *default* o no *default*. Es por esto que se proponen distintos modelos que van de lo más simple e interpretable hasta lo más complejo y menos interpretable. El ejemplo perfecto de que significa un modelo interpretable es el caso de la regresión lineal. En donde obtenemos un coeficiente para cada variable descriptiva y podemos formular resultados del tipo “un incremento de una unidad en la variable X_i representa un cambio de α en la variable Y ”.

Después de una breve investigación, la cual se cubrirá en la sección de cada método, se propone utilizar el siguiente conjunto de modelos:

Cuadro 3.8. Algoritmos de clasificación

Modelo
Regresión Logística
Regresión Ridge
Árbol de Decisión
Bosque Aleatorio
XGBoost
LightGBM



Para la implementación de los modelos utilizaremos principalmente las librerías de Scikit-Learn[6], XGBoost[36] y Lightgbm[7] de Python. Estas librerías de código abierto se desarrollaron específicamente para abordar los problemas de aprendizaje de máquina y hoy en día son consideradas como unas de las principales herramientas en esta área. Intentaremos usar notación de las mismas librerías para mantener congruencia con la implementación del trabajo.

Por último, es importante mencionar que no se intenta cubrir una construcción formal de cada método. El verdadero objetivo es describir de manera general cómo funcionan, por qué nos pueden servir y cuáles son sus principales consideraciones al momento de intentar aplicarlos. Sin embargo, se incluyen suficientes referencias para que el lector pueda profundizar en los conceptos más importantes o de mayor interés.

3.3.1. Regresión Logística

La familia de predictores lineales es una de las más comunes y conocidas. Muchos algoritmos utilizan métodos lineales por su habilidad de aprendizaje eficiente, por su fácil interpretación o intuición y por que se ajustan bien a muchos problemas clásicos. Para una introducción más completa, recomiendo el capítulo nueve del libro[29]. La regresión lineal es uno de los primeros métodos que permiten modelar la relación entre un conjunto de variables explicativas y una variable respuesta continua. Pensemos que tenemos un conjunto de k variables explicativas representadas por el vector $X = (X_1, X_2, \dots, X_k)$ y una variable respuesta Y . Este método nos permite expresar dicha relación en términos de una función lineal que depende únicamente de X de la forma

$$f(X) = \beta_0 + \sum_{i=1}^k \beta_i X_i \quad (3.1)$$

donde β_j s son parámetros desconocidos o coeficientes. Este modelo asume que la función de regresión $E(Y|X)$ es lineal o que el modelo es una aproximación razonable de Y . En la realidad lo que nosotros tenemos es el conjunto de entrenamiento $(x_1, y_1), \dots, (x_N, y_N)$ el cual podemos utilizar para estimar los parámetros β . El método de estimación más popular es el de mínimos cuadrados que busca minimizar la suma residual de cuadrados definida como:

$$RSS(\beta) = \sum_{i=1}^N (y_i - f(x_i))^2 = \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^k \beta_j X_{ij})^2 \quad (3.2)$$

Consultar Apéndice A.5.1 para mayor detalle. Esta es precisamente la función que buscamos minimizar en el problema de optimización. Entonces, la regresión lineal utiliza el método de mínimos cuadrados

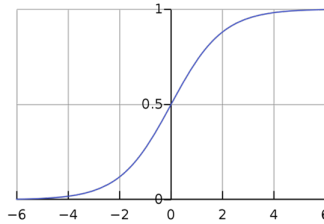
para encontrar una solución, es decir, los parámetros óptimos de la función lineal. Estos son los conceptos más importantes detrás de este modelo, los cuales tienen una gran influencia en casi todos los modelos posteriores, fuera de la familia de predictores lineales y también bajo el contexto de clasificación. Se puede continuar con este desarrollo en la sección 3.2 del libro [14]. Además, uno de los libros más completos acerca de este método es *Linear Regression Analysis* de Montgomery, para quien esté interesado en profundizar en el tema.

Como nuestro problema es de clasificación, donde la variable respuesta Y es binaria, tenemos que buscar hacer el salto a este contexto. Así surge la regresión logística, como una alternativa a la regresión lineal para poder pasar de problemas de predicción de una variable continua a un problema de clasificación de una variable categórica. Ahora Y puede tomar el valor de 0 o 1. Por lo tanto, quisiéramos interpretar $f(X)$ como $P(Y = 1|X)$ la probabilidad de que Y tome el valor de 1 dado un conjunto de características específicas. Entonces, el objetivo del modelo sería estimar esta probabilidad. Para lograr el salto es necesario introducir el concepto de las funciones sigmoide (*sigmoid functions*). En particular, la función logística que tiene la forma:

$$f(x) = \frac{1}{1 + \exp(-x)} : \mathbb{R} \rightarrow [0, 1]$$

La imagen de dicha función se comporta de la siguiente manera:

Figura 3.3. Gráfica de la función logística



De ahí el nombre de sigmoide, por su forma de “S”. Vemos que para valores grandes de x la función se acerca a 1, para valores pequeños se acerca a 0 y para valores cercanos a cero se acerca a $\frac{1}{2}$. Esta es una propiedad deseada para poder pensar e interpretarla como una probabilidad. Revisar el Apéndice A.5.3 para profundizar un poco más en este concepto.

Ahora que tenemos una función con estas características, podemos hacer la composición de una función lineal con la función logística de la forma $f_h(x) = f(h(x)) \in [0, 1]$. Con $h(x)$ una función lineal y $f(x)$ la función logística. Por lo tanto, para una observación (x_k, y_k) dada, denotamos

$$p(x_k) = f_h(x_k) = \frac{1}{1 + \exp(-h(x_k))} = \frac{1}{1 + \exp(-\beta_0 - \beta_1 x_k)}$$

como la probabilidad de que $y_k = 1$ y $1 - p_k$ como la probabilidad de que $y_k = 0$. Entonces, el error en la predicción se vería de la siguiente manera:

$$\begin{cases} p_k & \text{si } y_k = 1 \\ 1 - p_k & \text{si } y_k = 0 \end{cases}$$

Para lograr una función de pérdida monótona con respecto al error en la predicción podemos utilizar el logaritmo y combinar ambos casos,

definiendo la función de pérdida logarítmica como:

$$-y_k \log(p_k) - (1 - y_k) \log(1 - p_k)$$

Una función monótona es aquella que mantiene el orden dado, se puede revisar el Apéndice A.5.4 para más detalles. Por lo tanto, dado un conjunto de entrenamiento $S = (x_1, y_1), \dots, (x_m, y_m)$ el problema de optimización asociado a la pérdida logarítmica para la regresión logística sería minimizar la suma

$$-\sum_{k=1}^m (y_k \log(p_k) + (1 - y_k) \log(1 - p_k))$$

sobre el universo de parámetros de la función lineal (β_0, β_1) . La ventaja de este problema es que la función de pérdida es convexa, se puede revisar el Apéndice A.5.5. Por lo tanto, el problema de optimización se puede resolver utilizando métodos estándar. Además, tiene una gran similitud con el problema de estimación máximo verosímil.

Recapitulando, el objetivo del modelo es estimar $p_k = P(y_k = 1|x_k)$ con $f_h(x_k) = \frac{1}{1+\exp(-\beta_0-\beta_1 x_k)} \in [0, 1]$. Esta estimación se conoce como el *score* del modelo. El cuál nos dice la probabilidad de que la observación k pertenezca a la clase $Y = 1$ dado un conjunto de características específicas x_k . En nuestro contexto, el score significa la probabilidad de que un cliente, representado por una solicitud de crédito específica, caiga en *default*. Por lo tanto, una vez que tenemos los parámetros óptimos (β_0^*, β_1^*) , la función $f_h(x)$ está definida y podemos pronosticar un score para cada solicitud. Este es el resultado final del modelo. El concepto del *score* se extiende para todos los algoritmos de clasificación y es precisamente lo que se utiliza para poder medir el desempeño de los modelos.



Los principales supuestos de la regresión logística son:

1. Existe una relación lineal entre el valor de la función logística para cierto valor de la variable respuesta y cada una de las variables independientes.
2. No multicolinealidad, es decir, las variables independientes no están altamente correlacionadas entre ellas.
3. Ausencia de valores atípicos. Más que un supuesto, esto es un posible riesgo, pues ante la presencia de valores atípicos, las estimaciones del modelo se vuelven más inestables y menos precisas.
4. Observaciones independientes, es decir, cada observación del conjunto es independiente y no provienen de la misma muestra.

Se puede consultar el artículo[37] para una referencia rápida acerca de los principales supuestos de algoritmos de aprendizaje supervisado.

Estos son los principales conceptos detrás de este algoritmo de clasificación binaria. Se puede continuar con el desarrollo formal en la sección 9.3 del libro [29]. Por último, en el Apéndice A.6 se presentan algunos detalles sobre la implementación de cada algoritmo.

3.3.2. Regresión Ridge

Otra alternativa de un modelo lineal y una variación directa de la regresión lineal es la regresión ridge. Para introducir este modelo es necesario introducir el concepto de *shrinkage* (contracción o encogimiento), que trata de introducir una restricción que permite mejorar alguna condición del modelo. La regresión ridge “encoge” los

coeficientes de la regresión lineal imponiendo una penalización sobre su tamaño y por eso se considera un método de encogimiento. En específico la suma de residuales al cuadrado sufre la siguiente modificación.

$$\sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^k \beta_j X_{ij})^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (3.3)$$

Aquí $\lambda \geq 0$ es un parámetro que controla el magnitud del encogimiento, entre mayor valor de λ mayor es el encogimiento. También se le conoce como parámetro de complejidad, pues lleva a reducir varios coeficientes a cero. En particular, esto atiende el problema de la robustez de los coeficientes ante la multicolinealidad, es decir, mitigar el peso de coeficientes de variables correlacionadas. En general, esta penalización se puede considerar como un factor de sesgo que nos lleva a reducir la varianza del modelo. Revisar sección 3.4 del libro [14] para profundizar en este método.

Recordemos que nuestro problema es de clasificación y no de regresión, por lo que necesitamos un puente, al igual que en la regresión logística. Para lograrlo, primero se convierte la variable respuesta binaria en -1 y 1 y luego se trata como un problema de clasificación, generando predicciones entre -1 y 1 . A diferencia de la regresión logística, este algoritmo nos va a generar una distribución del *score* más suave o uniforme al no tener el efecto de la función exponencial. Además, ahora no tendremos la interpretación de la función de predicción como una probabilidad, sino más bien tendremos el efecto de los signos interpretando valores cercanos a 1 como positivos, cercanos a -1 como negativos y alrededor de 0 como un área gris. Si bien el uso de la regresión ridge bajo el contexto de

clasificación no es algo usual, será interesante ver su comparación contra la regresión logística y el resto de los modelos.

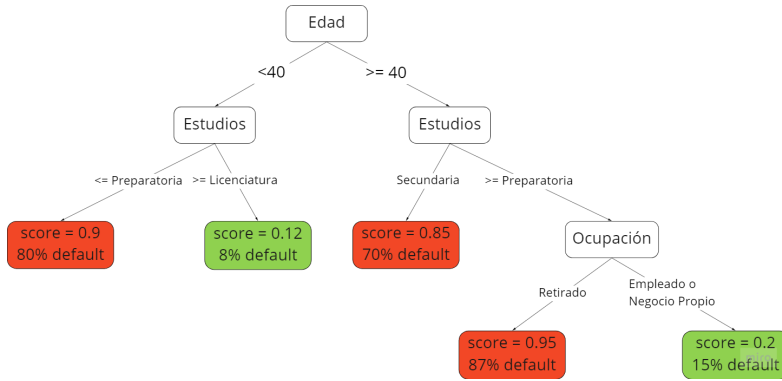
Por último, los supuestos de la regresión ridge son prácticamente los mismos que la regresión lineal, mismos que ya fueron descritos en la regresión logística y se resumen en linealidad, no multicolinealidad e independencia.

3.3.3. Árbol de Decisión

Hasta el momento, los dos modelos presentados buscan relaciones lineales entre las características y la variable objetivo. Por lo tanto, carecen de capacidad de encontrar relaciones no lineales y aquí es dónde surge la necesidad de buscar una estructura de aprendizaje distinta. El árbol de decisión busca ir partiendo los datos múltiples veces de acuerdo a una regla de corte, creando subconjuntos de datos que son representados como nodos y estos a su vez van formando una estructura similar a un árbol con ramas y hojas. Hay una gran variedad de estos algoritmos, pero nosotros usaremos la familia de modelos CART, recomendando el capítulo 9 del libro[14] para consultar una introducción más formal.

Una de las principales ventajas de un árbol de decisión es su alta interpretabilidad. Al poder visualizar la estructura del modelo de una forma gráfica, el entendimiento de qué aprendió y cómo está tomando decisiones se vuelve mucho más transparente. Veremos un ejemplo visual de la representación de este método para lograr mayor entendimiento.

Figura 3.4. Ejemplo de un árbol de decisión



En este ejemplo tenemos tres variables descriptivas, una discreta (edad), una categórica ordinal (estudios) y una categórica nominal (ocupación). Podemos observar cómo se va generando una partición para cada variable y esta a su vez genera una nueva rama del árbol. Tomando como ejemplo la primer rama (de izquierda a derecha) el árbol se puede leer de la siguiente manera: dentro del conjunto de observaciones con menos de 40 años y sin estudios superiores se tiene un nivel de *default* de 80%, por lo tanto un cliente con estas características tendrá un *score* = 0.9.

Otra de las grandes ventajas del árbol de decisión, es que casi no tiene supuestos que validar, esto permite aplicarlos con mayor libertad y en un mayor número de problemas. En comparación con los modelos lineales, estos modelos son robustos ante valores atípicos por la misma estructura de los mismos. Sin embargo, el árbol de decisión se considera un procedimiento de varianza alta, i.e. ante una pequeña variación en los datos se genera una diferencia importante en la estructura del árbol subyacente. Los próximos tres modelos son parte

de la familia de modelos basados en árboles, los cuáles buscan resolver algunas de las principales limitantes o problemas de los árboles de decisión.

3.3.4. Bosque Aleatorio

Para presentar este y los próximos algoritmos, es necesario introducir algunos conceptos clave *bootstrap*, *bagging* y *boosting*. No existe una traducción precisa, pero podríamos considerarlos como técnicas de remuestreo. Estas técnicas surgen a partir del concepto de *vox populi* o “sabiduría de las masas” cuya idea central consiste en afirmar que el conocimiento colectivo de un grupo diverso de personas es mayor al conocimiento de un único individuo. Por lo tanto, un esquema de votación entre un grupo diverso de personas nos daría mejor información que la decisión de un sólo individuo.

Bootstrap es un método para medir incertidumbre, muestreando del conjunto de entrenamiento. Surge bajo el contexto no paramétrico (no se supone ninguna estructura), como una técnica de remuestreo que estima la distribución de una población basada en muestro aleatorio con reemplazo. La idea central es tomar múltiples muestras de una población para estimar alguna característica, por ejemplo la media, introduciendo un factor de simulación que permite resolver el problema de tener “pocos” datos. Para profundizar en este concepto revisar la sección 7.11 y 8.2 del libro[14]. Bajo el contexto de los algoritmos, podemos pensar en entrenar varios modelos utilizando distintas muestras de datos para luego agruparlos. De ahí viene el concepto central de *bagging* o *bootstrap aggregation*. Esta técnica busca reducir la varianza de una función de predicción estimada. Dado que, los árboles de decisión tienen varianza alta y sesgo bajo, se convierten

en el perfecto candidato para aplicar dicha técnica y así surgen los *bagged trees*. En el contexto de problemas de clasificación podemos entender el efecto de esta técnica como un consenso de aprendices débiles (*weak learners*) independientes. Se puede consultar el artículo [35] para profundizar en estas técnicas.

Aunque este método incrementa la precisión de la predicción, empeora la interpretabilidad, pues ya no tenemos un árbol de decisión sino una especie de “árboles promedio”. Los bosques aleatorios surgen como una mejora a los *bagged trees* buscando reducir la correlación entre los árboles muestreados. También se construyen varios árboles sobre las muestras *bootstrap*, pero al construirlos, cada vez que se considera una separación se toma una muestra aleatoria de m predictores entre los p posibles y sólo se utiliza uno en cada separación. Esto para evitar correlación entre árboles y lograr una mayor reducción de varianza. Además, también evitamos que un predictor “fuerte” siempre domine. Luego de que tenemos un gran número de árboles, cada uno vota por la clase más popular y así obtenemos el bosque aleatorio. Para una introducción más formal a los bosques aleatorios consultar el artículo [16] o el capítulo 17 del libro [14].

En general, los modelos de ensamble son aquellos que utilizan una composición de modelos para generar un predictor final conjunto. **EL** bosque aleatorio es uno de los modelos de ensamble más sencillos e utilizados. Estos modelos tienen dos tareas principales: generar una población de aprendices base a partir del conjunto de entrenamiento y luego combinarlos para formar un predictor compuesto. Los próximos dos modelos también son parte de esta familia y también están basados en árboles de decisión como el aprendiz débil.

3.3.5. XGBoost

Ahora vamos a introducir el concepto de *boosting*. Esta técnica funciona similar que *bagging*, pero los árboles se construyen de manera secuencial. Cada iteración de un árbol utiliza la información del árbol anterior y una versión modificada del conjunto de datos original. Los próximos dos métodos son variantes del *boosting* con árboles, las cuales se han posicionado entre el grupo de modelos de última generación (“state-of-the-art”) para abordar problemas de clasificación dentro del aprendizaje supervisado. Hay dos factores de éxito en estos modelos, el uso efectivo de modelos estadísticos que capturan las relaciones complejas de los datos y la escalabilidad de los sistemas de aprendizaje que logran construir el modelo a partir de grandes conjuntos de datos.

XGBoost viene del concepto *gradient tree boosting*. Un método de *boosting* de árboles que utiliza el método de gradiente y búsqueda codiciosa (*greedy search*) para optimizar el resultado en cada paso del entrenamiento e ir definiendo la estructura de cada árbol. Adicional a esto, se utilizan dos técnicas para prevenir el sobre-ajuste. La primera y más relevante es la de *shrinkage*, que escala el peso en cada paso para reducir la influencia de cada árbol individual y permite que próximos candidatos mejoren el modelo. El artículo [36] presenta tanto el marco teórico de este método, como los detalles de la implementación para su uso. El sistema aquí propuesto es el mismo que se utiliza para el desarrollo de este trabajo. Así que esta es una de las mejores fuentes de información para profundizar en dicha técnica.

3.3.6. LightGBM

Una de las limitantes de los modelos de *gradient tree boosting*, como el caso de XGBoost, es que para cada variable descriptiva se hace una búsqueda casi exhaustiva sobre todos los posibles puntos de corte o separación para estimar la ganancia de información y tomar una decisión óptima. Esto puede llevar mucho tiempo de computo y ser poco eficiente para conjuntos de datos demasiado grandes. Así surge el método de *LightGBM* que busca acelerar el proceso de entrenamiento, sin sacrificar la precisión de las predicciones. Para lograr esto se utilizan dos técnicas, *Gradient-base One-Side Sampling* (GOSS) y *Exclusive Feature Bundling* (EFB). A gran escala, GOSS excluye instancias de datos con gradientes pequeños y sólo usa el resto para estimar la ganancia de información. Por otro lado, EFB busca reducir el número de variables agrupando característica mutuamente excluyentes usando un algoritmo de búsqueda codiciosa. El detalle de estas técnicas y método salen del alcance de este trabajo, pero en el artículo [7] se puede consultar la propuesta original de los autores, así como los principales detalles de la implementación. El sistema aquí propuesto es el mismo que se utiliza para el desarrollo de este trabajo. Así que esta es una de las mejores fuentes de información para profundizar en dicha técnica.

3.4. Métricas de evaluación

En esta sección explicaremos las principales métricas a considerar en los problemas de clasificación de aprendizaje supervisado. El objetivo es contar con una serie de métricas para medir y comparar el desempeño de los modelos y tener un criterio de optimalidad.

3.4.1. Contexto del problema

Recordemos que el objetivo principal del trabajo es utilizar los modelos para poder identificar si un nuevo cliente (representado por su solicitud de crédito) terminará incumpliendo en sus pagos o será un cliente legítimo. De entrada podemos establecer un punto de partida para saber si alguno de los modelos está mejorando la situación actual. Para esto es necesario introducir dos conceptos. La tasa de aceptación definida como el porcentaje de solicitudes que se aprueban del total de solicitudes que llegan al proceso de análisis. Y la tasa de *default* definida como el porcentaje de créditos que cayeron en *default* sobre el total de créditos otorgados. A partir de estos dos conceptos podemos definir el objetivo esperado de las siguientes tres maneras:

1. Logrando mejorar la tasa de aceptación manteniendo la tasa de *default*.
2. Logrando reducir la tasa de *default* manteniendo la tasa de aceptación.
3. Logrando mejorar la tasa de aceptación y reducir la tasa de *default*.

Desde este momento podemos percatarnos de que se busca un balance entre *default* y aceptación.

Como mencionamos anteriormente, los modelos de aprendizaje propuestos arrojan un *score* que para la mayoría de los casos (excepto la regresión ridge) se puede interpretar como la probabilidad de que un cliente caigan en un incumplimiento. Por lo tanto, el modelo per se no toma decisiones, sino nos devuelve dicho *score* y nosotros tenemos que tomar una decisión de aprobar o rechazar. Esta decisión es una de las últimas y más importantes tareas en el desarrollo de un modelo de

clasificación. Básicamente, tenemos que establecer un *score* de corte para decidir que todo lo que esté al nivel o por arriba pertenecerá a la clase de *default* y viceversa para la clase de clientes legítimos. Aquí surgen algunas dudas importantes como:

- ¿Cómo medir el desempeño de cierto punto de corte?
- ¿Por qué sólo establecer un punto de corte y tomar decisiones binarias?
- ¿Una vez definida una métrica de desempeño, cómo definir el *score* de corte óptimo?

Estas son sólo algunas de las preguntas que pueden surgir al analizar este reto, pues este por si sólo constituye otro problema de optimización. Se buscará ir dando respuesta a este tipo de preguntas e justificar las decisiones que se vayan tomando. En la práctica, vemos muchos modelos que establecen el punto de corte en .5 solamente por ser el punto medio. Esto es un error grave, pues esta decisión siempre va ligada al contexto y objetivo puntual del problema.

3.4.2. Matriz de confusión y métricas subyacentes

Para poder entender qué pasa una vez que se define un punto de corte vamos a introducir la matriz de confusión. Recordemos que por convención, la clase de interés se denota como la clase “positiva”, en nuestro caso la clase de *default*. Por otro lado, la otra clase es la clase “negativa”, en nuestro caso la clase de cliente buenos o que no cayeron en *default*. La matriz de confusión busca comparar los resultados del modelo contra lo que en realidad sucedió utilizando cuatro métricas. Veamos un ejemplo visual para entenderlo mejor.

Figura 3.5. Ejemplo de una matriz de confusión

		Predicciones	
		Positivos	Negativos
Observado	Positivos	Verdaderos positivos (TP)	Falsos negativos (FN)
	Negativos	Falsos positivos (FP)	Verdaderos negativos (TN)

Los verdaderos positivos (*true positives* o TP) son observaciones que el modelo clasificó como *default* y en realidad si cayeron en *default*. Los falsos positivos (*false positives* o FP) son observaciones que el modelo clasificó como *default* y en realidad eran legítimas. Y así correspondientemente para los verdaderos negativos (*true negatives* o TN) y para los falsos negativos (*false negatives* o FN). Podemos interpretar los dos cuadros verdes como los aciertos del modelo y los dos cuadros rojos como los errores para cada clase. Es importante mencionar que, en nuestro problema, cada tipo de acierto o error nos dice cosas distintas. Por ejemplo, no es lo mismo otorgar un crédito a una persona que va a incumplir (FN), que no prestarle a un cliente legítimo (FP). A partir de esta matriz se definen las siguientes métricas de desempeño:

- *Accuracy* (exactitud) es la proporción de predicciones correctas sobre el total de predicciones, definida como $\frac{(TP+TN)}{(TP+FP+FN+TN)}$.
- *Precision* (precisión) es la proporción de predicciones correctas sobre el total de predicciones positivas, definida como $\frac{TP}{(TP+FP)}$.
- *Recall* (sensibilidad, cobertura o nivel de captura) es la proporción de predicciones correctas sobre el total de positivos, definida como $\frac{TP}{(TP+FN)}$. También se conoce como tasa de verdaderos positivos (*true positive rate* o TPR).

- *Specificity* (especificidad) es la proporción de predicciones correctas sobre el total de negativos, definida como $\frac{TN}{(TN+FP)}$. También se conoce como tasa de verdaderos negativos (*true negative rate* o TNR).

Si sólo medimos la exactitud, estaríamos obteniendo la proporción de aciertos, pero no tendríamos noción de la magnitud del error para cada clase. Si sólo medimos la precisión, estaríamos obteniendo el acierto y el error de las predicciones de *default*, sin observar qué pasó con las predicciones de no *default*. Si sólo medimos el *recall*, obtendríamos cuántos clientes malos detectó y no detectó el modelo, sin conocer el resultado para los clientes legítimos. Si sólo medimos la *specificity*, obtendríamos el complemento del *recall*. Entonces, podemos concluir que no conviene utilizar sólo una métrica, pues en realidad nos interesa conocer la historia completa. Por otro lado, podemos pensar que de acuerdo al objetivo de nuestro problema lo que buscamos es maximizar la precisión y al mismo tiempo el *recall*. Es decir, maximizar la captura de *default* y maximizar la pureza de dicha captura. Este será la discusión de la próxima sección.

3.4.3. Balance entre *precision* y *recall*

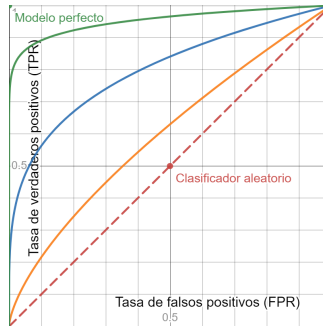
Buscar maximizar ambas métricas de manera simultanea genera un problema. Si quisiéramos maximizar la captura, simplemente clasificaríamos todos los casos como positivos, pero esto provocaría que la precisión disminuyera hasta el nivel original del conjunto. Si quisiéramos maximizar la precisión, seguramente estaríamos sacrificando el nivel de captura. Si tomamos las 10 observaciones con mayores *scores*, esperamos tener una mayor precisión que si tomamos 100. Por lo tanto, este es un intercambio y debemos buscar un balance.

Una métrica muy popular que incorpora esta idea se llama *F1 – score*. Se define como $F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$, la media armónica de estas métricas. Esto nos permite agrupar ambas métricas, castigando valor extremos. De hecho, hay una versión más general que nos permite asignarle un peso o factor de importancia a cualquiera de las dos métricas y se define como $F_\beta = (1 + \beta)^2 \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}$. Ahora que ya tenemos una métrica más robusta nos enfrentamos con otro problema, hasta este momento todas las métricas presentadas dependen de un punto de corte específico. ¿Será necesario calcular estas métricas para distintos puntos de corte o cómo podríamos agrupar el desempeño de un modelo bajo todos los posibles puntos de corte? Este será la discusión de la próxima sección.

3.4.4. Curvas ROC, PRC y métricas subyacentes

Para poder tomar en consideración todos los posibles puntos de corte podemos pensar en una función continua que depende del punto de corte y regresa alguna métrica. De esta manera podemos construir curvas para representar todo el posible universo de puntos de corte. Sería natural pensar en el área bajo la curva como la posible métrica de agrupación que logre resumir el desempeño sobre todos los posibles puntos de corte. La curva ROC (*Receiver operating characteristic*) se crea graficando la tasa de verdaderos positivos (TPR o *recall*) contra la tasa de falsos positivos (FPR o 1-TNR). Veamos un ejemplo para entenderlo mejor.

Figura 3.6. Ejemplos de curvas ROC



Podemos interpretar la curva ROC como el balance o *trade-off* entre costo (FPR) y beneficio (TPR). El espacio va de $[0,1]$ en ambos ejes, por lo que el área total es de uno, entonces el área bajo la curva igual irá de $[0,1]$. Por un lado, el modelo de predicción perfecto generaría el punto verde en las coordenadas $(0,1)$ pues estaría capturando el 100 % de los casos positivos sin ningún error o falso positivo. Por otro lado, un modelo aleatorio (lanzar una moneda al aire) se espera que genere puntos sobre la línea diagonal roja (conocida como la línea de no discriminación). Esta línea divide el espacio en dos y es claro que su área es de 0.5. Por lo tanto, podemos pensar que un buen clasificador generaría una curva por encima de la diagonal con un área > 0.5 y un mal clasificador estaría sobre la diagonal o por debajo con un área ≤ 0.5 . Podríamos pensar que la curva amarilla corresponde a un modelo regular, la azul a uno bueno y la verde a uno excelente.

Entonces para generar una curva solamente hay que graficar TPR contra FPR a lo largo del segmento $[0,1]$. En la práctica se utiliza la regla trapezoidal para aproximar la integral y obtener el área bajo la

curva. El área bajo la curva ROC se conoce como *AUC (area under the curve) ROC*, nosotros la denotaremos como AROC para efectos de este trabajo.

Con el objetivo de no depender únicamente de una sola métrica vamos a introducir una medida más. Después de introducir la curva ROC, sería natural replicar el concepto para comparar directamente la precisión contra el *recall*. Así surge la curva PRC (*precision-recall curve*), que describe de manera gráfica el *trade-off* entre ambas métricas a lo largo de todos los posibles puntos de corte. Ahora buscamos valores altos para cada eje y por lo tanto un área bajo la curva también cercana a 1, pero usaremos otra medida de agrupación para resumir el comportamiento de la curva. Se define la precisión promedio (*average precision*) como $AP = \sum_n (R_n - R_{n-1})P_n$ el promedio de la precisión alcanzado en cada punto de corte ponderado por el incremento en el *recall* con respecto al punto anterior. Nosotros la denotaremos como APRC para efectos de este trabajo.

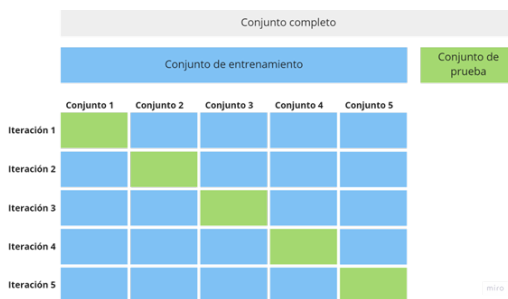
Ahora si tenemos un par de métricas que nos permite medir el desempeño de un modelo en términos del nivel de captura, la precisión y el error en la captura. Estas dos métricas en conjunto representan nuestro criterio de optimalidad para poder discernir entre modelos. Se puede consultar el artículo [34] o el contenido de [26] para seguir explorando las distintas métricas de desempeño de los modelos de aprendizaje supervisado.

3.5. Entrenamiento, validación y monitoreo del modelo

3.5.1. Validación Cruzada

El concepto de validación cruzada parte de la idea de separar el conjunto de datos, pero en vez de hacerlo una sola vez de manera estática, se convierte en algo dinámico. La idea es dividir el conjunto en $k \in \mathbb{N}$ partes iguales e iterar el proceso de entrenamiento y validación k veces. Por ejemplo, si tomamos $k = 5$, haríamos una partición del conjunto en 5 pedazos e iterar para cada pedazo. En la primera iteración, seleccionamos el primer subconjunto como conjunto de validación y entrenamos con los otros cuatro. En la segunda iteración, seleccionamos el segundo para validar y entrenamos con el resto. Y así sucesivamente hasta realizar las cinco iteraciones. Con esto logramos entrenar el modelo cinco veces, con cinco conjuntos distintos y validar cinco veces con cinco conjuntos distintos. Esto nos genera cinco resultados distintos partiendo de un sólo conjunto inicial. Es importante resaltar que la partición del conjunto inicial se realiza por medio de un muestreo aleatorio sin reemplazo. Es decir, si tenemos 25 observaciones, se toman de manera aleatoria cinco observaciones para cada grupo. Vamos a ver una representación visual de esta partición del conjunto de datos.

Figura 3.7. Representación visual de la validación cruzada



Fuente: Basada en el contenido de [25]

Podemos observar cómo dividimos el mismo conjunto de entrenamiento y mantenemos intacto el conjunto de prueba. Este es un procedimiento muy útil para generar resultados más robustos y poder medir variación entre los mismos. Este concepto será clave en todo el proceso de construcción del modelo.

Este procedimiento se puede realizar para distintas k , en nuestro caso utilizaremos $k = 5$, al ser lo estándar. Haciendo uso de este método, logramos utilizar un mismo conjunto para entrenar y validar. También se suele agregar la condición de estratificación con respecto a la variable objetivo dentro del proceso de separación de cada parte. Se puede consultar el artículo [20] o el contenido en [25] para más información.

3.5.2. Búsqueda de hiper-parámetros óptimos

Utilizando el concepto anterior realizaremos una búsqueda de los hiper-parámetros óptimos para cada clasificador. Los hiper-parámetros son parámetros que no se aprenden directamente del proceso de aprendizaje, sino que más bien se pasan como argumentos al modelo

en cuestión. Este proceso de búsqueda consiste en tomar un estimador o modelo específico, definir el espacio de hiper-parámetros, definir un método de búsqueda usando validación cruzada y una métrica de optimalidad. Para definir el espacio de parámetros hay que declarar un rango de posibles valores para cada uno de los parámetros de interés. Luego, el algoritmo entrena el modelo con todas las combinaciones de parámetros posibles y nos quedamos con la configuración que nos arroja los mejores resultados de acuerdo a la métrica de optimalidad. Nosotros vamos a utilizar un método de búsqueda exhaustivo llamado *GridSearchCV*. Se pueden consultar la referencia [27] para mayor detalle de la implementación. Por otro lado, en el Apéndice A.6 se describen los hiper-parámetros y el espacio de búsqueda para cada algoritmo.

3.5.3. Monitoreo y mejoras del modelo

Una vez que construimos un modelo y queremos utilizarlo para tomar decisiones es necesario estar validando su utilidad de manera constante. Es por ello que el monitoreo del modelo posterior a la puesta en producción es una de las tareas críticas para el correcto uso del mismo. Básicamente, monitorear que el modelo está arrojando resultados consistentes y “buenos” con respecto a una o varias métricas. Este paso suele omitirse en la teoría, pero es parte vital para demostrar el poder y utilidad de los modelos. La manera más directa de monitoreo el desempeño es medir el cambio en las métricas de evaluación que se utilizaron en la construcción del modelo usando nuevos datos que hemos logrado recolectar. Otro tipo de monitoreo más granular puede venir de un análisis de sensibilidad, cambios en la importancia o pesos de las variables o cambios poblacionales. Tenemos que pensar que el monitoreo del modelo, nos dará una perspectiva de

posibles mejoras o errores y esto nos llevará a iniciar de nuevo el proceso de entrenamiento, validación y prueba. Recordar que el desarrollo de un modelo es un proceso iterativo. A pesar de que el tema sale del alcance del trabajo quisiera dejar este artículo de referencia [22] para el lector que quiera profundizar en este asunto.

Capítulo 4

Resultados y Discusión

Una vez que definimos el problema, el objetivo del trabajo, los datos, las técnicas, los modelos y las métricas de evaluación estamos listos para iniciar con la construcción y validación de los modelos. Como se describió en el capítulo anterior, ~~casi~~ en cada paso del proceso de preprocesamiento, entrenamiento y ajuste del modelos se puede tomar más de un camino. Una buena estrategia para **discernir** es probar los distintos caminos e ir eligiendo el que genera mejores resultados. Aquí entra el método de validación cruzada como técnica de búsqueda y las métricas de validación como criterio de optimalidad. Esto nos permitirá explorar y elegir el camino óptimo para construir los modelos. Después, analizaremos los resultados a detalle y concluiremos con la construcción de un modelo de toma de decisión.

4.1. Definiendo la ruta óptima de preprocesamiento

En esta sección abordamos cada paso del proceso de preprocesamiento, siguiendo el mismo orden presentado en la sección 3.2.

4.1.1. Tratamiento de datos faltantes

De acuerdo a la descripción realizada en la sección 3.2.1, tenemos dos opciones de tratamiento, eliminar las observaciones que contienen algún dato faltante o aplicar el tratamiento descrito. Si quisiéramos eliminar observaciones perderíamos el 40 % de los datos. Por lo tanto, procederemos a aplicar el tratamiento descrito. Entonces, a partir de este momento ya no tenemos ningún dato faltante y podemos continuar con el preprocesamiento.

4.1.2. Codificación de variables no numéricas o categóricas

Recordemos que la implementación de los algoritmos sólo aceptan variables numéricas como entrada. Por lo tanto es necesario codificar las variable no numéricas. Siguiendo el tratamiento de codificación sin combinar clases, descrito en la sección 3.2.2, obtenemos los siguientes resultados sobre el conjunto de prueba.

Modelo	AROC	APRC
Regresión Logística	0.6622	0.4078
Regresión Ridge	0.7590	0.4878
Árbol de Decisión	0.6714	0.3967
Bosque Aleatorio	0.8034	0.5555
XGBoost	0.8067	0.5680
LGBM	0.8162	0.5970

Cuadro 4.1. Resultados aplicando la codificación sin agrupar clases

Esta tabla nos presenta el valor del área bajo la curva ROC (AROC) y la precisión promedio (APRC) para cada uno de los seis algoritmos de clasificación. Es importante mencionar que para generar esta tabla, primero se utilizó el conjunto de entrenamiento para “entrenar” los modelos. Después, se utilizó el conjunto de prueba para realizar predicciones y calcular ambas métricas. Por lo tanto, la tabla nos está diciendo cómo se desempeña cada algoritmo sobre nuevas observaciones. Se usará este mismo formato de tabla para comparar los resultados a lo largo de toda la sección 4.1.

~~Ahora si pasemos a revisar los resultados.~~ Recordemos que ambas métricas van de cero a uno y mientras más cercanas a uno mejor. Observamos que los tres modelos de ensamble presentan los mejores resultados. La regresión logística y el árbol de decisión son los dos peores modelos. La regresión ridge **está medio** de estos dos grupos. Aunque no se ve en la tabla, es importante mencionar que al haber aplicado la codificación directa (sin combinar clases) pasamos de tener 105 a 290 variables descriptivas. Esto es una clara evidencia del aumento exponencial de las dimensiones del conjunto, provocado por

el método de creación de variables dummies.

Ahora vamos a aplicar el tratamiento completo, es decir, combinando clases para las variables categóricas con más de cinco clases.

Modelo	AROC	APRC
Regresión Logística	0.6612	0.4057
Regresión Ridge	0.7767	0.5117
Árbol de Decisión	0.6666	0.3966
Bosque Aleatorio	0.8006	0.5631
XGBoost	0.8029	0.5798
LGBM	0.8051	0.5758

Cuadro 4.2. Resultados aplicando la codificación después de agrupar clases

Podemos observar que los resultados se mantuvieron casi constantes. La regresión ridge mejoró y el LGBM empeoró un poco. Lo importante a mencionar es que logramos reducir la dimensionalidad de 290 a 126 variables. Es decir, le quitamos mucha complejidad al problema sin sacrificar desempeño. Esto es suficiente evidencia como para concluir que este proceso brinda una mejora en el preprocesamiento y decidimos utilizarlo de aquí en adelante. Por lo tanto, hasta este momento ya se aplicó el tratamiento de datos faltantes, la agrupación de clases para ciertas variables y la codificación de variables no numéricas. Esto quiere decir que los tratamientos posteriores parten de este conjunto modificado.

4.1.3. Transformación de variables

Recordemos que los modelos lineales son muy sensibles a observaciones atípicas. Entonces, la hipótesis es que al estandarizar o aplicar alguna otra transformación equivalente dichos algoritmos tendrán mejores resultados. Para el resto de los modelos, prácticamente todos basados en árboles, no esperamos ver un cambio importante.

Comenzamos realizando la estandarización de todas las variables. Recordemos que después de la codificación todas nuestras variables son numéricas.

Modelo	AROC	APRC
Regresión Logística	0.7674	0.4997
Regresión Ridge	0.7770	0.5103
Árbol de Decisión	0.6773	0.4028
Bosque Aleatorio	0.7927	0.5549
XGBoost	0.8029	0.5803
LGBM	0.8175	0.5798

Cuadro 4.3. Resultados aplicando estandarización

Observamos que sólo la regresión logística tuvo una mejora considerable de su desempeño, para el resto de métodos no hubo gran diferencia. Por lo tanto, sólo tendría sentido aplicar la estandarización para la regresión logística.

Ahora revisaremos los resultados de aplicar la mezcla de transformaciones descrita al final de la sección 3.2.3.

Modelo	AROC	APRC
Regresión Logística	0.7725	0.5048
Regresión Ridge	0.7824	0.5122
Árbol de Decisión	0.6587	0.3862
Bosque Aleatorio	0.7957	0.5526
XGBoost	0.8084	0.5660
LGBM	0.8065	0.5752

Cuadro 4.4. Resultados aplicando la mezcla de transformaciones

Como era de esperar vemos que sólo los métodos lineales presentaron un mejor desempeño. Esto reafirma que la mezcla de transformaciones superó la simple estandarización y que será necesario comenzar a dar tratamientos personalizados dependiendo el tipo de algoritmo. Por lo tanto, procederemos a aplicar la mezcla de transformaciones para las dos regresiones y no transformar los datos para el resto de modelos.

4.1.4. Selección de variables

Como se describió al final de la sección 3.2.4, aplicando el método de votación para la selección de variables, se eliminaron 56 variables y obtenemos un conjunto reducido. Aplicando el mismo preprocesamiento definido hasta el momento, pero sobre el conjunto reducido obtenemos:

Modelo	AROC	APRC
Regresión Logística	0.7818	0.5415
Regresión Ridge	0.7832	0.5325
Árbol de Decisión	0.6646	0.3922
Bosque Aleatorio	0.7927	0.5762
XGBoost	0.7927	0.5585
LGBM	0.7990	0.5765

Cuadro 4.5. Resultados aplicando el método de votación para la selección de variables

Podemos observar que utilizando esta técnica las dos regresiones mejoraron en ambas métricas. El árbol de decisión mejoró muy poco y el resto de los modelos se mantuvieron constantes o empeoraron un poco. Es importante mencionar que los algoritmos basados en árboles y métodos de ensamble (bosques, XGBoost y LGBM) realizan un proceso intrínseco de selección de variables. Entonces, no es necesario aplicar este tipo de métodos explícitos. Por lo tanto, sólo nos conviene hacer una selección de variables previa para los modelos lineales.

Hasta este momento, hemos decidido aplicar el tratamiento de datos faltante, la combinación de clases y la codificación de variables no numéricas para todos los métodos. Además, para las dos regresiones decidimos aplicar la mezcla de transformaciones y luego el método de votación para la selección de variables. Por otro lado, como era de esperar el Árbol de Decisión se desempeña muy por debajo de los otros tres métodos basados en árboles y ensambles, por lo que podemos dejarlo fuera de la contienda. Entonces, los resultados óptimos hasta el momento son:

Modelo	AROC	APRC
Regresión Logística	0.7818	0.5415
Regresión Ridge	0.7832	0.5325
Bosque Aleatorio	0.8006	0.5631
XGBoost	0.8029	0.5798
LGBM	0.8051	0.5758

Cuadro 4.6. Resultados óptimos hasta el momento

4.1.5. Balanceo de datos

Ahora procederemos a explorar el balanceo del conjunto de datos. Recordemos que hasta este momento tenemos un conjunto de tratamientos para los modelos lineales y otro para el resto de los modelos. Por lo anterior, todos los posibles métodos de balanceo se harán para cada conjunto por separado. Otra consideración importante es que sólo en caso de observar una mejora considerable en el desempeño de los modelos procederemos a incorporar dicha técnica al tratamiento completo. Esto último debido a que el método general de balanceo implica la creación de observaciones sintéticas, es decir, observaciones artificialmente creadas y no observadas a priori. Todas las técnicas propuestas buscan aumentar el número de observaciones de la clase minoritaria, que en nuestro caso corresponde a la clase de *default*, sobre el conjunto de entrenamiento.

Comenzamos aplicando la técnica de *Random Over Sampling* para balancear el conjunto de entrenamiento descrita en la sección 3.2.5.

Modelo	AROC	APRC
Regresión Logística	0.7709	0.5003
Regresión Ridge	0.7834	0.5117
Bosque Aleatorio	0.7914	0.5583
XGBoost	0.8152	0.5766
LGBM	0.7987	0.5709

Cuadro 4.7. Resultados aplicando *Random Over Sampling*

Podemos observar que no se presenta una mejora importante en ninguno de los modelos. De hecho, fuera del XGBoost, se observa una caída en el desempeño de los algoritmos.

Ahora pondremos a prueba la técnica de *Synthetic Minority Over-sampling Technique* (SMOTE) descrita en el Apéndice A.4.

Modelo	AROC	APRC
Regresión Logística	0.7750	0.5162
Regresión Ridge	0.7825	0.5113
Bosque Aleatorio	0.7942	0.5622
XGBoost	0.7958	0.5538
LGBM	0.7930	0.5789

Cuadro 4.8. Resultados aplicando SMOTE

En este caso, todos los modelos empeoraron.

Por último, probaremos la técnica de *Borderline-SMOTE* descrita en el mismo Apéndice A.4.

Modelo	AROC	APRC
Regresión Logística	0.7718	0.4989
Regresión Ridge	0.7762	0.4974
Bosque Aleatorio	0.8008	0.5646
XGBoost	0.8058	0.5737
LGBM	0.7971	0.5668

Cuadro 4.9. Resultados aplicando *Borderline*-SMOTE

Podemos observar que tampoco hubo una mejora considerable para ningún modelo. Es importante mencionar que estos métodos sólo modifican el conjunto de entrenamiento y en específico el grupo de observaciones de la clase de *default*. Entonces, esto posiblemente sólo esté mejorando la captura de dicha clase al momento de entrenar. Pero si esto no nos lleva a un mejor desempeño sobre el conjunto de prueba, quiere decir que sólo estaríamos generando un sobre-ajuste. Además, el hecho de que nuestro problema no está “demasiado” desbalanceado puede ser otro motivo por el cual el efecto de estas técnicas no sea tan notorio. Por lo tanto, decidimos no aplicar ningún tipo de balanceo al conjunto de entrenamiento.

4.2. Entrenamiento y ajuste de modelos

En este momento ya logramos encontrar el conjunto de técnicas para diseñar el preprocesamiento óptimo para cada modelo. Vamos a recapitular el proceso, tomando en cuenta que a partir del paso cuatro sólo se aplicó el tratamiento para los modelos lineales.

1. Se trataron los datos faltantes.
2. Se combinaron las clases de variables categóricas con alta

cardinalidad.

3. Se codificaron las variables no numéricas.
4. Se transformaron las variables. En este momento todas las variables son numéricas.
5. Se seleccionaron variables usando el método de votación.

Para este momento de la construcción, ya habían pasado un par de meses desde que se construyó el conjunto de datos. Haciendo una actualización se lograron clasificar 303 nuevas observaciones, aproximadamente un 12% de datos nuevos. Por lo tanto, con la intención de incorporar los datos más recientes y incrementar considerablemente el conjunto de datos se decidió extender la venta de observación y agregar estas nuevas observaciones. Esto nos obliga a volver a definir la ruta óptima de preprocesamiento. Además, esperamos ver un cambio con respecto al desempeño de los modelos presentado hasta el momento. Aplicando el preprocesamiento anterior y con la actualización del conjunto de datos obtenemos los siguientes resultados.

Modelo	AROC	APRC
Regresión Logística	0.7536	0.4797
Regresión Ridge	0.7727	0.5044
Bosque Aleatorio	0.7837	0.5205
XGBoost	0.7760	0.5157
LGBM	0.7877	0.5170

Cuadro 4.10. Resultados después de actualizar el conjunto de datos

Observamos que hubo una caída generalizada en el desempeño de los modelos. Al haber actualizado el conjunto de datos perdimos la comparación directa con la primera parte de la experimentación, pues ya no son los mismos datos. Por lo anterior, realizamos una exploración rápida para volver a encontrar el preprocesamiento óptimo, siguiendo exactamente la misma lógica descrita anteriormente. El resultado fue el mismo, pero sin la selección de variables para los métodos lineales. Parece ser que la incorporación de esas 303 observaciones llevó a incrementar la capacidad predictiva de algunas variables. Es decir, usaremos todas las variables para todos los algoritmos, sólo que para las dos regresiones se aplica la mezcla de transformaciones. Entonces, los resultados del preprocesamiento óptimo bajo el conjunto de prueba final se ven así.

Modelo	AROC	APRC
Regresión Logística	0.7640	0.5029
Regresión Ridge	0.7843	0.5208
Bosque Aleatorio	0.7839	0.5311
XGBoost	0.7760	0.5157
LGBM	0.7877	0.5170

Cuadro 4.11. Resultados finales del preprocesamiento óptimo

Vamos a seleccionar cuatro algoritmos “ganadores” para profundizar en el análisis de desempeño y simplificar las comparaciones. Pensando en las posibles interacciones de los modelos y viendo la similitud de los resultados, nos quedaremos con los dos modelos lineales y los dos modelos de *boosting* con árboles. De esta forma vamos a lograr abarcar dos familias de modelos distintos y dos estructuras distintas de cada familia, esto le dará mucha riqueza al

análisis.

Una vez que definimos el tratamiento óptimo para los datos, es necesario realizar la búsqueda de hiper-parámetros óptimos. Como se mencionó en la sección 3.5.2, vamos a utilizar el método de *GridSearchCV*. El método consiste en definir un universo de posibles hiper-parámetros y hacer una búsqueda exhaustiva utilizando validación cruzada para cubrir todas las posibles combinaciones de parámetros. Además, vamos a utilizar el área bajo la curva ROC (AROC) como métrica de optimalidad para decidir de manera automática que configuración de parámetros es la óptima. Utilizando el espacio de hiper-parámetros descritos en el Apéndice A.6 se realizó la búsqueda y obtuvimos una configuración óptima para cada método. Tomando dichos parámetros óptimos y re-entrenando sobre el conjunto de entrenamiento completo obtenemos los siguientes resultados sobre el conjunto de prueba.

Modelo	AROC	APRC
Regresión Logística	0.7933	0.5431
Regresión Ridge	0.7832	0.5177
XGBoost	0.7876	0.5394
LGBM	0.7844	0.5346

Cuadro 4.12. Resultados con hiper-parámetros óptimos

Observamos una mejora generalizada en el desempeño de los modelos, en especial para la regresión logística. Algo muy interesante es que con la configuración óptima de hiper-parámetros, los cuatro modelos están muy parejos. De manera sorprendente, la regresión logística se está desempeñando por encima de los dos modelos de

ensambles.

Ahora que tenemos los resultados finales para cada modelo vamos a analizar las métricas más a detalle. Observamos que los valores del área bajo la curva ROC (AROC) varían entre (0.78, 0.80), esto nos dice que los cuatro modelos tienen buena capacidad para distinguir entre la clase de *default* y no *default*. Por otro lado, la precisión promedio ponderada (APRC) varía entre (0.51, 0.55), aunque esta no es tan alta, recordemos que se está promediando sobre todo el rango de posibles puntos de corte. Únicamente observando estas métricas, diríamos que el mejor modelo es la regresión logística, luego XGBoost, luego LGBM y por último la regresión Ridge. Por lo tanto, hasta el momento tenemos cuatro modelos que lograr distinguir entre la población de clientes buenos y malos, dos modelos con estructuras lineales y dos modelos con métodos de ensamble utilizando árboles como esquema de aprendizaje. Recordemos que cada modelo genera un *score* por observación. El *score* se puede interpretar como la probabilidad de que dicha observación pertenezca a la clase de *default*.

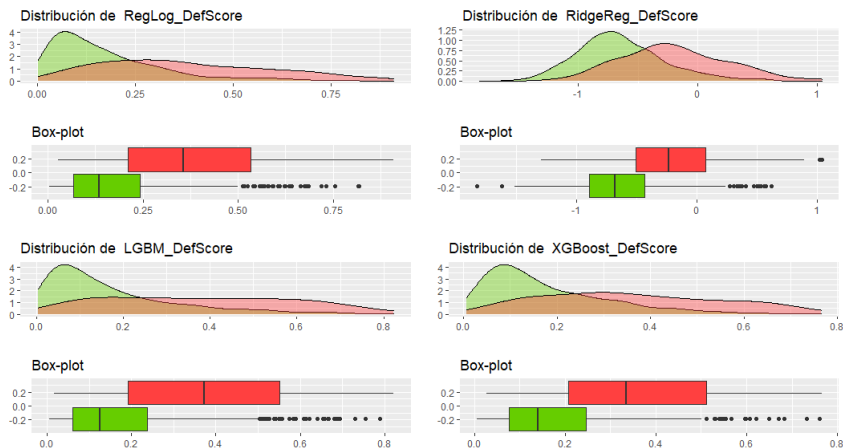
4.3. Analizando los resultados a detalle

Para entender mejor el comportamiento de cada modelo y algunas interacciones entre ellos, procederemos a analizar la distribución del *score* para cada modelo, medir correlaciones entre ellos, analizar la importancia o peso de las variables explicativas, entre otras cosas.

Primero, observamos la distribución del *score* de *default* para cada modelo, separando por la variable objetivo para comparar los *scores* de los clientes malos (en rojo) contra los buenos (en verde). El resultado

esperado sería ver una acumulación de clientes malos a la derecha (*scores* altos) y viceversa.

Figura 4.1. Distribuciones del *score* de cada modelo



Observamos un comportamiento muy similar para tres modelos y uno distinto para la regresión Ridge. Para los tres modelos con comportamiento similar tenemos una clara acumulación de clientes buenos con *scores* bajos de *default* y una cola que va desapareciendo a partir del punto de corte en 0.5. Esto es ~~excelente~~ evidencia de que dichos modelos lograron aprender la estructura detrás de los clientes que no cayeron en incumplimiento. Por otro lado, la distribución de clientes que si cayeron en *default* parece más uniforme, manteniendo la presencia a lo largo de todo el eje horizontal. Esto nos habla de una deficiencia de los modelos al momento de identificar un cliente malo, aunque en el extremo derecho si vemos una clara acumulación de clientes malos. Es interesante ver que el *score* de la regresión Ridge tiene una distribución más “normal” que el resto y los valores del *score*

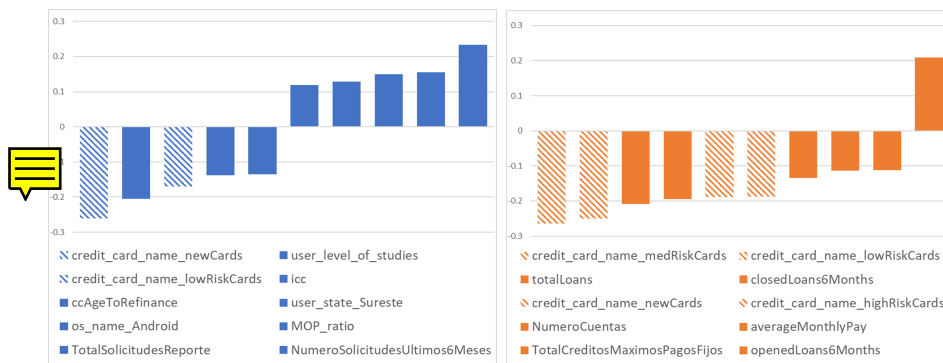
van entre menos dos y dos. Recordemos que este modelo es una modificación directa de una regresión lineal. Y que el puente entre el problema de regresión y el de clasificación se hace transformando la variable objetivo a valores de menos uno y uno correspondientemente.



A diferencia de la regresión logística, no hay una función exponencial detrás de la estimación del *score*. Por lo tanto, es de esperar que la distribución del score tenga este comportamiento más suave y que la escala del score sea distinta a los demás.

Ahora analizamos la importancia de las variables para cada modelo. En el caso de las regresiones observando sus coeficientes y para los modelos de ensamble utilizando la estimación del factor de importancia.

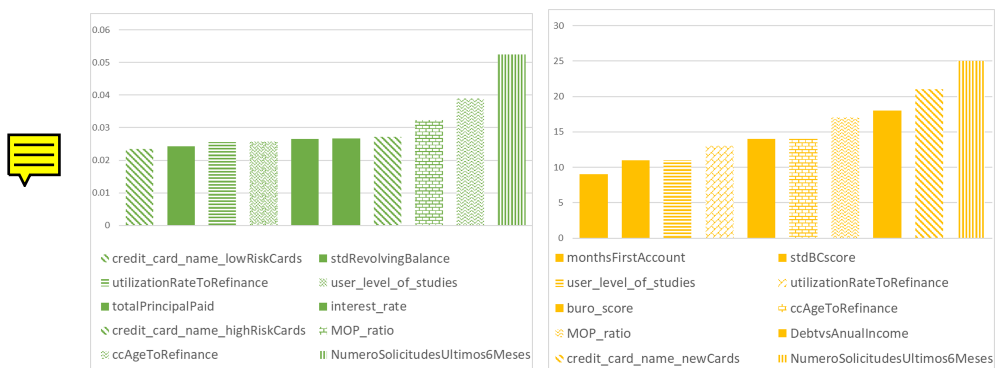
Figura 4.2. Coeficientes más influyentes para las regresiones



En el caso de las regresiones (azul = Logística y naranja = Ridge), observamos coeficientes positivos y negativos. Como estamos observando los coeficientes del *score* de la clase positiva o *default*, podemos interpretar cada coeficiente como el peso que tiene dicha

variables en el cálculo del *score* de pertenencia a la clase de clientes malos. Por lo tanto, coeficientes positivos nos hablan de una relación negativa en la clasificación de un cliente y viceversa. Vemos algunas coincidencias en las variables. Ambos modelos consideran el tipo de tarjeta de crédito y el número de búsquedas en Buró en los últimos seis meses como variable muy importantes.

Figura 4.3. Variables más influyentes para los modelos de ensamble



En el caso de los modelos de ensamble (verde = XGBoost y amarillo = LGBM), observamos el peso que tiene cada variable en el proceso de aprendizaje. Por lo tanto, aquí no tenemos una relación positiva o negativa, sino la influencia total de cada característica. Vemos que ambos modelos tienen la variable *NumeroSolicitudes6Meses* como la de mayor peso, reforzando lo descrito para las dos regresiones. En general, esperábamos una relación estrecha entre estos dos métodos, pues sabemos que parten de las mismas bases.

Haciendo la intersección de los cuatro modelos, identificamos

aquellas variables en donde la mayoría de los modelos (al menos tres de cuatro) coincidieron en las variables más y menos influyentes.

Variables más importantes	Variables menos importantes
<i>creditCardName</i>	<i>onTimeLoans</i>
<i>NumeroSolicitudes6Meses</i>	<i>userGender</i>
<i>userLevelOfStudies</i>	<i>TotalSaldosActuales</i>
<i>ccAgeToRefinance</i>	<i>activeLoans</i>
<i>MOP ratio</i>	<i>maritalStatus</i>

Cuadro 4.13. Variables más y menos relevantes

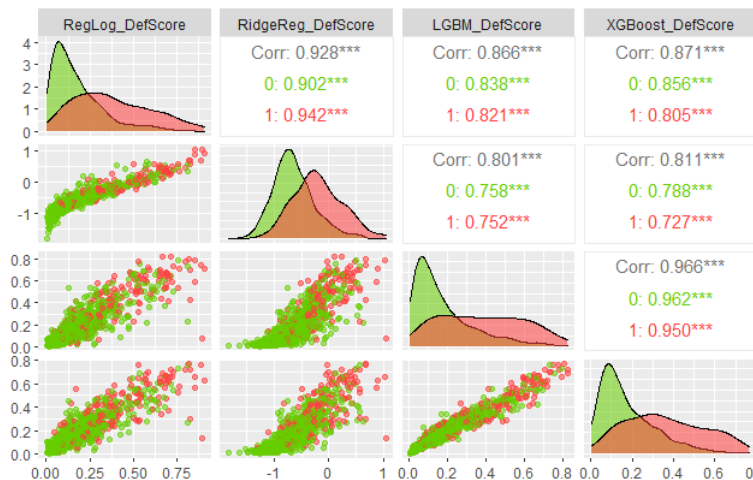
Las variables más importantes nos hacen bastante sentido, pues desde el análisis exploratorio se vio la relación con la variable objetivo. Además, las cinco nos hablan de características relevantes del cliente. Por ejemplo, el número de solicitudes reportadas por Buró (*NumeroSolicitudes6Meses*) nos dice cuántas veces se solicitó el historial crediticio de dicho cliente por parte de instituciones financieras. Esto nos habla del movimiento crediticio del cliente fuera de la empresa y podemos conjeturar que una persona con demasiadas consultas sería un comportamiento sospechoso. Por otro lado, algunas variables irrelevantes también **hacen** sentido de manera individual. Por ejemplo, el género del usuario o el total de saldos en otras cuentas, que por si solas no nos dicen mucho.

Es importante mencionar que para los dos modelos de ensamble se pueden obtener pesos de cero para algunas variables. Esto quiere decir que dichas variables no aportaron información en el proceso de aprendizaje. En realidad, para el modelo XGBoost 58 variables tuvieron un peso igual a cero y para el modelo LGBM 41 variables. Esto reafirma la capacidad intrínseca de dichos modelos de hacer una

selección de variables y prescindir de dicho método al momento del preprocesamiento.

Ahora vamos a comparar los cuatro modelos entre si, midiendo correlación y graficando la intersección de los *scores* por pares.

Figura 4.4. Comparación de los *scores* de los modelos



Como era de esperar los dos modelos lineales tienen una alta correlación lineal positiva y los dos modelos de ensemble igual. Esto debido a que familia de modelos parte de bases muy similares, por lo tanto sería extraño esperar comportamientos muy distintos. Vemos mayor variabilidad entre familias de modelos, entre una regresión y un modelo de ensemble. Observando las gráficas de la intersección de los modelos por pares (toda la esquina inferior izquierda) podemos entender un poco más de dicha interacción, pues vemos la acumulación de buenos cuando ambos modelos tienen *scores* bajos y viceversa. Esto nos lleva a pensar en qué pasaría si en vez de elegir un sólo modelo

“ganador”, nos quedaríamos con dos y utilizaríamos su interacción para construir zonas de decisión más precisas. Esto sería transformar un problema de partición de una recta a uno de un espacio bidimensional. Por lo tanto, para el resto del análisis nos quedaremos con los dos mejores modelos, pero con estructuras distintas entre si. Es decir, la regresión logística y el XGBoost.

4.4. Definiendo un modelo de toma de decisión

Hasta el momento únicamente tenemos un *score* por modelo para cada observación, pero lo que nos interesa es tomar una decisión asociada a una posible acción a aplicar a cada solicitud de crédito. Para entender mejor el tipo de decisiones y las condiciones que buscamos, describiremos una serie de hipótesis asociadas.

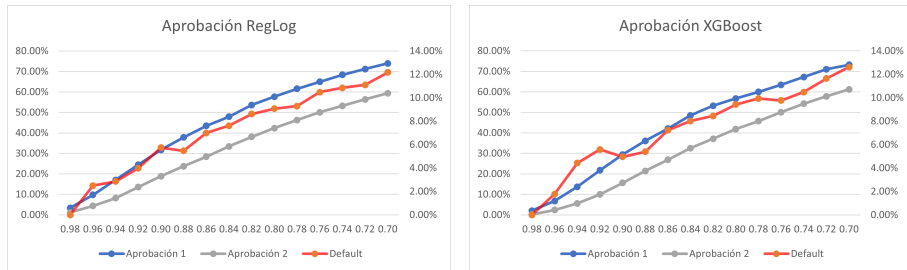
1. Una solicitud de crédito se puede aprobar o rechazar. Al día de hoy, todas las solicitudes pasan por un proceso de revisión manual para tomar una decisión.
2. De acuerdo al objetivo del negocio, buscamos un balance entre la tasa de aceptación y la tasa de *default*. Prácticamente la mayor aceptación posible con el menor nivel de incumplimiento.
3. Buscamos que el modelo logre automatizar decisiones de aprobación o rechazo. Es decir, queremos aceptar automáticamente a los clientes con mayor probabilidad de ser clientes buenos y viceversa para los rechazos.
4. Para los casos de incertidumbre, el modelo simplemente llevaría a un análisis posterior, manteniendo el flujo original de la

solicitud. Estos casos suelen ser el área gris, cuando los *scores* están alrededor del 0.5.

A partir de las hipótesis anteriores, podemos concluir que necesitamos construir tres zonas de decisión, una zona de aprobación, una de rechazo y todo lo demás sería la zona de análisis. Haremos una revisión a detalle de la posible zona de aprobación, buscando aceptar al mayor número de clientes “buenos” manteniendo una tasa de *default* muy baja (establecida por el negocio). Por otro lado, una revisión a detalle de la posible zona de rechazo, buscando declinar los casos más problemáticos con una tasa de falsos positivos baja (establecida por el negocio). Para robustecer el análisis, se procesaron todas las solicitudes que llegaron a análisis y se rechazaron. Es decir, se modelaron las solicitudes que se rechazaron manualmente. Esto nos permitirá medir los niveles de aceptación que se hubieran logrado sobre un conjunto mucho más amplio, al incluir las que solicitudes que se rechazaron.

Comenzamos revisando la zona de aprobación. Queremos encontrar un *score* de corte para decidir aprobar un cliente a partir de su probabilidad de pertenencia a la clase de no *default*. Hasta el momento habíamos utilizado el score de *default*, pero podemos pensar en el score de no *default* como el complemento.

Figura 4.5. Zona de aprobación para cada modelo



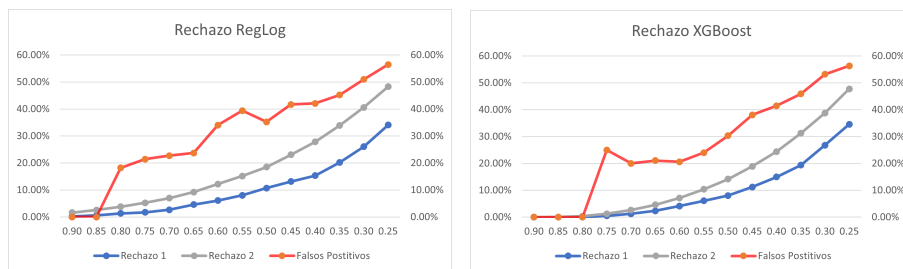
Vamos a explicar las gráficas. El eje horizontal corresponde al posible *score* de corte que podemos tomar para decidir aprobar una solicitud. Sólo visualizamos del 0.98 al 0.70, pues es la zona de mayor interés para tomar una decisión de aprobación. El eje vertical-izquierdo (eje principal) corresponde a la tasa de aprobación y el eje vertical-derecho (eje secundario) a la tasa de *default*. Por otro lado, la línea azul (Aprobación 1) corresponde al nivel de aprobación del conjunto de prueba logrado al tomar cada posible *score* de corte. Después, la línea gris (Aprobación 2) nos dice el nivel de aprobación logrado sobre el conjunto de solicitudes que se rechazaron durante el proceso de análisis. Por último, la línea roja representa el nivel de *default* del conjunto de prueba.

Observamos una clara diferencia entre los niveles de aprobación uno y dos para ambos modelos, siempre aprobando por debajo para el conjunto de solicitudes rechazadas. Esto es un buen resultado, ya que no quisiéramos tener la misma aprobación para las solicitudes en realidad se aceptaron contra las que en realidad se rechazaron. Por otro lado, observamos que la tasa de *default* va aumentando casi de forma lineal. Esto nos permite entender de manera más visual el

balance que hay que buscar entre la tasa de aprobación y la tasa de *default*, siempre existe este intercambio y dependerá de la aversión al riesgo y otras consideración del negocio por establecer el *score* de corte óptimo. Por ejemplo, si fijamos la tasa de *default* en 10 %, la regresión logística tiene unas tasas de aprobación de aproximadamente 63 % y 48 % y el XGBoost de 64 % y 51 % correspondientemente.

Ahora revisamos la zona de rechazo. Buscamos fijar un *score* de corte para decidir rechazar un cliente a partir de su probabilidad de pertenencia a la clase de *default*.

Figura 4.6. Zona de rechazo para cada modelo



Estas gráficas tienen una estructura muy similar a las anteriores, el eje horizontal corresponde al posible *score* de corte que podemos tomar, pero ahora para decidir rechazar una solicitud. El eje vertical-izquierdo (eje principal) corresponde a la tasa de rechazo y el eje vertical-derecho (eje secundario) a la tasa de falsos positivos. Recordemos que podemos interpretar la tasa de falsos positivos como el error en la zona de rechazo, es decir, qué porcentaje de los clientes que se rechazaron en realidad eran “buenos”. La línea azul (Rechazo 1) representa el nivel de rechazo del conjunto de prueba logrado al tomar cada posible *score* de corte. La línea gris (Rechazo 2) igual, pero sobre

el conjunto de solicitudes rechazadas en el proceso de análisis. Por último, la línea roja representa el nivel de falsos positivos del conjunto de prueba.

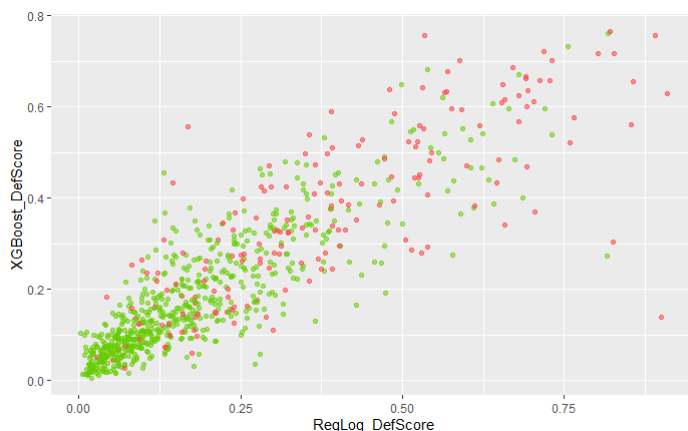
En general, observamos un peor desempeño de los modelos en esta zona. Primero, en cuanto a las tasa de rechazo, se invierte la relación manteniendo la tasa del conjunto de solicitudes rechazadas siempre por arriba del conjunto de prueba. Esto es bueno, ya que estamos declinando más solicitudes de las que en realidad se rechazaron en el proceso de análisis. Sin embargo, la diferencia no es tan marcada como en la zona de aprobación. Por otro lado, la tasa de falsos positivos tiene un comportamiento bastante inestable al inicio (para *scores* altos). Esto reafirma una deficiencia en el aprendizaje de clientes malos. Este será un tema a considerar en las conclusiones y trabajo a futuro.

Una buena noticia es que el costo asociado con un error en la zona de rechazo es menor que en la zona de aprobación. Es decir, si aprobamos un cliente malo el costo sería toda la deuda pendiente por pagar al momento del *default* y cuando rechazamos un cliente bueno el costo sería únicamente la ganancia esperada que dejamos ir. Esto quiere decir, que aunque los modelos tienen un bajo desempeño en la zona de rechazo, se compensan con un buen desempeño en la zona de aprobación la cuál es la más delicada. Esto nos llevaría a pensar que al momento de definir el modelo de toma de decisión, tendremos mayor confianza en la zona de aprobación y seremos más cautelosos en la zona de rechazo. Además, esto también es un incentivo para buscar mayor detalle en las zonas de aprobación y rechazo. Por este motivo, pasaremos a analizar la interacción de estos dos modelos y cómo esto

nos puede llevar a aumentar los niveles de precisión, disminuir los errores y por ende aumentar la confianza en la decisión final.

Vamos a volver a observar la interacción directa entre ambos *scores*, distinguiendo los clientes malos y buenos con mayor detalle. A partir de este punto sólo utilizaremos el *score* de pertenencia a la clase de *default*.

Figura 4.7. Comparación de los dos *scores* finales



Como ya habíamos adelantado, observamos una clara acumulación de clientes “buenos” (en verde) cuando ambos *scores* son bajos. En general, podemos identificar una fuerte correlación lineal positiva entre ambos modelos y sólo algunos puntos de contraste donde un modelo difiere del otro. Todo esto nos da aún más detalle de cómo se podría construir un modelo de toma de decisión que hiciera una partición en este espacio bidimensional aprovechando al máximo dicha interacción. Para esto podemos transformar esta gráfica en una matriz de contingencia, donde medimos como va cambiando el nivel de pureza en cada región del espacio.

Figura 4.8. Matriz de contingencia del *default* acumulado

LR / XGB	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.9	0.95	1
0.05	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%	0.36%
0.10	0.49%	0.97%	1.09%	1.34%	1.46%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%	1.58%
0.15	0.49%	1.09%	1.58%	2.31%	3.04%	3.28%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%	3.41%
0.20	0.49%	1.46%	2.31%	3.53%	4.50%	4.87%	5.23%	5.35%	5.35%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%	5.47%
0.25	0.49%	1.46%	2.55%	4.62%	5.84%	6.45%	7.18%	7.30%	7.30%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%	7.42%
0.30	0.49%	1.46%	2.80%	4.99%	6.45%	7.42%	8.52%	8.64%	8.76%	8.88%	8.88%	8.88%	9.00%	9.00%	9.00%	9.00%	9.00%	9.00%	9.00%	9.00%
0.35	0.49%	1.46%	2.92%	5.11%	7.06%	8.39%	9.98%	10.83%	11.19%	11.44%	11.44%	11.56%	11.68%	11.68%	11.68%	11.68%	11.68%	11.68%	11.68%	11.68%
0.40	0.49%	1.46%	2.92%	5.11%	7.30%	8.76%	10.46%	11.80%	12.65%	13.02%	13.50%	13.63%	13.75%	13.75%	13.75%	13.75%	13.75%	13.75%	13.75%	13.75%
0.45	0.49%	1.46%	2.92%	5.11%	7.30%	9.00%	10.83%	12.53%	13.63%	14.36%	14.84%	14.96%	15.09%	15.09%	15.09%	15.09%	15.09%	15.09%	15.09%	15.09%
0.50	0.49%	1.46%	2.92%	5.11%	7.30%	9.00%	10.83%	12.53%	13.87%	14.84%	15.45%	15.69%	15.82%	15.82%	15.82%	15.82%	15.82%	15.82%	15.82%	15.82%
0.55	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.14%	14.96%	16.18%	16.91%	17.40%	17.52%	17.52%	17.52%	17.52%	17.52%	17.52%	17.52%	17.52%
0.60	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.14%	14.96%	16.30%	17.52%	18.13%	18.37%	18.73%	18.73%	18.73%	18.73%	18.73%	18.73%	18.73%
0.65	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.26%	15.21%	16.55%	18.25%	18.98%	19.22%	19.59%	19.59%	19.59%	19.59%	19.59%	19.59%	19.59%
0.70	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.38%	15.45%	16.79%	18.49%	19.46%	19.71%	20.32%	20.32%	20.32%	20.32%	20.32%	20.32%	20.32%
0.75	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.59%	20.07%	20.80%	20.92%	20.92%	20.92%	20.92%	20.92%	20.92%
0.80	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.83%	20.44%	21.17%	21.29%	21.29%	21.29%	21.29%	21.29%	21.29%
0.85	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.83%	20.56%	21.29%	21.53%	21.65%	21.65%	21.65%	21.65%	21.65%
0.90	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.83%	20.56%	21.41%	21.65%	21.78%	21.78%	21.78%	21.78%	21.78%
0.95	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.83%	20.56%	21.41%	21.65%	21.78%	21.78%	21.78%	21.78%	21.78%
1	0.49%	1.46%	2.92%	5.11%	7.30%	9.25%	11.19%	13.50%	15.57%	16.91%	18.61%	19.83%	20.56%	21.41%	21.65%	21.78%	21.78%	21.78%	21.78%	21.78%

El eje vertical corresponde a la variación de los puntos de corte para la regresión logística y el eje horizontal para el XGBoost. Lo que estamos midiendo en cada cuadro es el nivel de *default* acumulado al realizar la intersección de ambos cortes. Por ejemplo, el primer cuadro (esquina superior izquierda) nos dice que si tomamos todas las solicitudes de crédito donde ambos *scores* son ≤ 0.05 obtenemos una tasa de *default* de 0.36 %. Esta matriz nos permite explorar las tres posibles zonas de decisión de manera visual. Podríamos pensar que el área verde es potencial zona de aprobación, el área amarilla de análisis y el área roja de rechazo. Pero como se mencionó en las hipótesis del modelo de toma de decisión, es necesario definir los límites bajo los parámetros de riesgo de la empresa. Se realizó un ejercicio con los tomadores de decisión y expertos de la empresa observando las distintas métricas. Además, se transformó la aprobación en montos colocados y la tasa de *default* en pérdida. Por otro lado, se transformó el rechazo en monto declinado y la tasa de falsos positivos en costo de oportunidad. A partir de dicho ejercicio se definieron las siguientes zonas de decisión con sus respectivos parámetros o límites. Definiremos s_1 como el *score* de *default* de la regresión logística y s_2 como el *score*

de *default* del XGBoost.

1. Aprobación automática: se va a aprobar de manera automática todas aquellas solicitudes donde

$$(s_1 \leq 0.033) \cup (s_2 \leq 0.022) \cup ((s_1 \leq 0.068) \cap (s_2 \leq 0.042))$$

Esto quiere decir que aprobamos de manera automática cuando cualquiera de las tres condiciones ocurre. La regresión logística (s_1) tienen un *score* ≤ 0.033 . El XGBoost (s_2) tienen un *score* ≤ 0.022 . O ambos tienen un *score* por debajo de ciertos límites correspondientes. Estos puntos de corte se definieron buscando la mayor captura posible con nivel de *default* menor al 2 %.

2. Aprobación semi-automática: vamos a sugerir la aprobación incentivando un análisis confirmatorio para las solicitudes con

$$(s_1 \leq 0.078) \cup (s_2 \leq 0.061) \cup ((s_1 \leq 0.175) \cap (s_2 \leq 0.147))$$

Esta zona se definió buscando la mayor captura posible con un nivel de *default* menor al 5 %.

3. Rechazo automático: se va a rechazar de manera automática todas las solicitudes donde

$$(s_1 \geq 0.684) \cup (s_2 \geq 0.592)$$

Esta zona se definió buscando la mayor captura posible con un nivel de *default* de 70 %.

4. Rechazo semi-automático: vamos a sugerir el rechazo incentivando un análisis confirmatorio para las solicitudes con

$$(s_1 \geq 0.500) \cup (s_2 \geq 0.550) \cup ((s_1 \geq 0.440) \cap (s_2 \geq 0.450))$$

Esta zona se definió buscando la mayor captura posible con un nivel de *default* de 60 %.

5. Análisis: vamos a mandar a revisión manual el resto de la solicitudes. Es decir, todo lo que no cayó en ninguna de las cuatro zonas anteriores.

Es importante recalcar que cada zona es excluyente y se fue creando en el orden presentado, para que las zonas semi-automáticas sean complementarias a las automáticas y la zona de análisis sea el complemento de las cuatro.

Ahora vamos a analizar la distribución de la población de prueba por zona decisión.

Decisión del modelo	% de captura	Nivel de <i>default</i>
<i>Aprobación automática</i>	7.18 %	1.69 %
<i>Aprobación semi-automática</i>	32.85 %	4.81 %
<i>Análisis</i>	49.03 %	26.80 %
<i>Rechazo semi-automático</i>	7.30 %	60 %
<i>Rechazo automático</i>	3.65 %	70 %

Cuadro 4.14. Distribución del conjunto de prueba por zona

Podemos observar que la zona de aprobación automática está logrando capturar 7.18 % del conjunto de prueba acumulando un nivel de *default* de 1.69 %. Si quisiéramos replicar el nivel de captura con un sólo modelo obtendríamos niveles de *default* de aproximadamente 3.4 % y 5 % respectivamente. Esto nos habla de la mejora drástica que implica utilizar la interacción de los dos modelos. También podemos observar que para las zonas de rechazo los niveles de captura caen

bastante, esto reafirma el bajo desempeño de los modelos en identificar solicitudes que cayeron en incumplimiento.

Por último, vamos a incluir el análisis sobre la población de rechazo, para hacer una simulación más realista del impacto del modelo. Se replicó el modelo de decisión para todas las solicitudes rechazadas y se sumaron los resultados para obtener las siguientes métricas.

Tipo de decisión	Aprobación (%)	Rechazo (%)	Total (%)
<i>Decisión automática</i>	6.59 %	9.33 %	15.92 %
<i>Decisión semi-automática</i>	21.93 %	9.70 %	31.63 %
<i>Decisión manual</i>	52.45 %	52.45 %	52.45 %

Cuadro 4.15. Distribución del conjunto de prueba y rechazo

Esto quiere decir que con el uso del modelo se hubieran logrado automatizar más del 15 % de las decisiones, se hubieran optimizado 31.63 % y se hubieran mantenido manuales el 52.45 %.

Conclusiones

El presente trabajo nos permitió hacer uso de métodos estadísticos-computacionales, en específico algoritmos de aprendizaje supervisado, para resolver la medición del riesgo crediticio para una fintech mexicana. A partir de una metodología sólida para la construcción de un modelo, se logró hacer uso de conceptos teóricos y técnicos para transformar información descriptiva de un cliente que está solicitando un crédito en decisiones de aprobación, análisis o rechazo. De manera puntual el uso del modelo hubiera permitido:

- Automatizar aproximadamente 15 % de las decisiones divididas en dos zonas.
 - Logrando mantener la tasa de *default* por debajo del 2% sobre el conjunto de aprobación automática.
 - Rechazando automáticamente un grupo con una tasa de *default* por encima del 70 %.
- Optimizar un poco más del 30 % de las decisiones.
- Reducir el volumen de solicitudes que llegan a análisis en un 50 %. Tomando en consideración que el tiempo de revisión manual puede llevar varios minutos y el tiempo de procesamiento del modelo un par de segundos.

Para aquellos lectores que gusten profundizar más en la metodología utilizada en este trabajo, se pueden consultar tres referencias interesantes. La primera, una tesis[23] de carácter más formal. La segunda[1] y tercera[32], un par de casos de éxito de empresas que están revolucionando la manera de cuantificar el riesgo crediticio. Es muy interesante y satisfactorio encontrar puntos en común, pero también puntos de contraste y área de oportunidad, dentro de estas tres referencias. En general, los modelos comparten buena parte de la estructura del proceso de construcción y varían en cuanto al cómo utilizarlos para la toma de decisión. Por otro lado, se puede validar que las siguientes ideas presentadas como parte de este trabajo parecen brindar un enfoque innovador al problema: el método para la creación de una ruta óptima de preprocesamiento, el uso de dos modelos para la construcción del modelo de toma de decisión y el análisis y definición de múltiples zonas de decisión.

Por último, vamos a mencionar algunas consideraciones pensando en continuar con el desarrollo de este trabajo.

1. Retomar la discusión sobre utilizar variables de clientes recurrentes y el impacto de tener un mezcla con clientes nuevos.
2. Profundizar en la limitante de sólo usar solicitudes de créditos aprobadas como la población para el entrenamiento y validación de los modelos. Este problema se conoce como *reject inference* y se puede consultar la referencia [1] para más información.
3. Atender con mucho mayor detalle el hecho de que estamos trabajando un problema desbalanceado, es decir, tenemos mayor proporción de clientes legítimos que de clientes malos.

4. En general, las técnicas de preprocesamiento se personalizaron y esto generó obstáculos para poderlas aplicar bajo el contexto de validación cruzada. Esto fue una limitante técnica, que se sobrepasó muy tarde y no se pudo aplicar en el desarrollo práctico de este trabajo.
5. El diseño del modelo de toma de decisión es una tarea casi artesanal, pues depende 100 % del contexto y objetivo del problema en cuestión. Esto implica de manera directa que hay una infinidad de caminos para plantear y desarrollar este paso.
6. Este trabajo es sólo la parte inicial del desarrollo e implementación de una solución de negocio. Una vez que tenemos un modelo como este, es necesario “ponerlo en producción” para que esté integrado al flujo operacional. Después de esto vienen las etapas de monitoreo y próximas iteraciones. Estas otras partes son críticas para lograr una mejor y mayor adopción de estas técnicas en la vida cotidiana.

Bibliografía

- [1] Ben Taborsky. 2020. “Exploration for Machine Learning at Affirm”. *Affirm Tech Blog*. Julio 6, 2020. <https://tech.affirm.com/exploration-for-machine-learning-at-affirm-632ea5930f54>
- [2] Buro de Crédito. 2022. “Glosario”. Consultado en septiembre, 2022. <https://www.burodecredito.com.mx/glosario.html>
- [3] Christoph Molnar. 2022. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2da Edición. <https://christophm.github.io/interpretable-ml-book/index.html>
- [4] Deepanshu Bhalla. “CREDIT RISK: VINTAGE ANALYSIS”. *Listen Data*. Consultado en septiembre 2022. <https://www.listendata.com/2019/09/credit-risk-vintage-analysis.html>
- [5] Deepanshu Bhalla. “Weight of Evidence (WOE) and Information Value (IV) Explained”. *Listen Data*. Consultado en noviembre 2021. <https://www.listendata.com/2015/03/weight-of-evidence-woe-and-information.html>

- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel *et al.* 2011. “Scikit-learn: Machine Learning in Python”. *Journal of Machine Learning Research*, volumen 12.
- [7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, T. Liu *et al.* 2017. “Lightgbm: A highly efficient gradient boosting decision tree”. *Advances in Neural Information Processing Systems*, volumen 30.
- [8] Huda. 2021. “How to handle Categorical variables?”. *Medium*. 23 mayo, 2021. <https://medium.com/geekculture/how-to-handle-categorical-variables-7c1ee198c55c>
- [9] Investopedia. 2022. “Credit”. Última modificación marzo 7, 2022. <https://www.investopedia.com/terms/c/credit.asp>
- [10] Investopedia. 2022. “Credit Risk: Definition, Role of Ratings, and Examples”. Última modificación marzo 15, 2022. <https://www.investopedia.com/terms/c/creditrisk.asp>
- [11] Investopedia. 2022. “Default: What It Means, What Happens When You Default, Examples”. Última modificación septiembre 6, 2022. <https://www.investopedia.com/terms/d/default2.asp>
- [12] James D. McCaffrey. 2020. “Should You Normalize and Encode Data Before Train-Test Splitting, or After Splitting?”. mayo 27, 2020. <https://jamesmccaffrey.wordpress.com/2020/05/27/should-you-normalize-and-encode-data-before-train-test-splitting-or-after-splitting>
- [13] Jeff Desjardins. 2017. “The History of Consumer Credit in One Giant Infographic”. *Visual*

- Capitalist*. <https://www.visualcapitalist.com/history-consumer-credit-one-infographic>
- [14] Jerome Friedman, Trevor Hastie y Robert Tibshirani. 2008. *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. 2da Edición. Springer.
 - [15] Krishna Venkatraman. 2022. “What is holding back financial inclusion”. *The Fintech Times*. Edición 45, julio 2022. https://issuu.com/fintechtimes/docs/the_fintech_times_digital_edition_45
 - [16] Leo Breiman. 2001. *Random Forest*. Statistics Department University of California Berkeley, California.
 - [17] Lyn C. Thomas, David B. Edelman y Jonathan N. Crook. 2002. *Credit Scoring and Its Applications*. Society for Industrial and Applied Mathematics, Philadelphia.
 - [18] Lyn C. Thomas. 2009. *Consumer Credit Models: Pricing, Profit, and Portfolios*. Oxford University Press, New York.
 - [19] Mao Bing-Huan, Han Hui and Wang Wen-Yuan. 2005. “Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning”. *Advances in Intelligent Computing. ICIC 2005. Lecture Notes in Computer Science*, vol 3644. Springer, Berlin, Heidelberg.
 - [20] Mohammed Alhamid. 2020. “What is Cross-Validation? Testing your machine learning models with cross-validation”. *Towards Data Science* 24 diciembre, 2020. <https://towardsdatascience.com/what-is-cross-validation-60c01f9d9e75>

- [21] Jorge Nocedal y Stephen J. Wright. 2006. *Numerical Optimization*. 2da Edición. Springer Series in Operations Research and Financial Engineering, Nueva York.
- [22] Philip Tannor. 2022. “How to Monitor ML Models in Production”. *DeepChecks* blog. Consultado el 16 septiembre, 2022. <https://deepchecks.com/how-to-monitor-ml-models-in-production>
- [23] Rodrigo Magaña. 2016. “Desarrollo de un Modelo de Comportamiento Crediticio para la Administración de una Cartera de Tarjeta de Crédito”. Tesis Licenciatura, ITAM.
- [24] Sanford Weisberg. 2001. “Yeo-Johnson Power Transformations”. Department of Applied Statistics, University of Minnesota.
- [25] Scikit Learn. “3.1. Cross-validation: evaluating estimator performance”. Consultado en enero 2021. <https://scikit-learn.org/stable/modules/cross-validation.html>
- [26] Scikit Learn. “3.3. Metrics and scoring: quantifying the quality of predictions”. Consultado en enero 2021. https://scikit-learn.org/stable/modules/model_evaluation.html
- [27] Scikit Learn. “3.2. Tuning the hyper-parameters of an estimator”. Consultado en enero 2021. https://scikit-learn.org/stable/modules/grid_search.html#grid-search
- [28] Shipra Saxena. 2020. “Here’s All you Need to Know About Encoding Categorical Data (with Python code)”. *Analytics Vidhya*. 13 agosto, 2020. <https://www.analyticsvidhya.com/blog/2020/08/types-of-categorical-data-encoding>

- [29] Shai Shalev-Shwartz y Shai Ben-David. 2014. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, Nueva York.
- [30] Songhao Wu. 2020. “Multicollinearity in Regression: Why it is a problem? How to check and fix it”. *Towards Data Science*. 18 mayo, 2020. <https://towardsdatascience.com/multi-collinearity-in-regression-fe7a2c1467ea>
- [31] Spectral. 2022. “How Web3 Creditworthiness can create Financial Inclusion”. Consultado el 16 de septiembre, 2022. <https://blog.spectral.finance/how-web3-creditworthiness-can-create-financial-inclusion>
- [32] Spectral. 2022. “A Deeper Look at the MACRO Score (Part 1)”. Consultado el 16 de septiembre, 2022. <https://blog.spectral.finance/a-deeper-look-at-the-macro-score-part-one>
- [33] Sundar Krishnan. 2018. “Variable Selection using Python - Vote based approach”. *Medium*. 7 mayo, 2018. <https://sundarstyles89.medium.com/variable-selection-using-python-vote-based-approach-faa42da960f0>
- [34] Tavish Srivastava. 2020. “11 Important Model Evaluation Metrics for Machine Learning Everyone should know”. *Analytics Vidhya*. Última modificación 5 agosto, 2020. <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics>
- [35] Thomas G. Dietterich. 2000. “Ensemble Methods in Machine Learning”. In: *Multiple Classifier Systems*. MCS 2000. Lecture Notes in Computer Science, vol 1857. Springer, Berlin, Heidelberg.

- [36] Tianqi Chen y Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System”. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. Nueva York.
- [37] Timmothy Tan. 2020. “Back to Basics: Assumptions of Common Machine Learning Models”. *Towards Data Science*. 8 junio, 2020. <https://towardsdatascience.com/back-to-basics-assumptions-of-common-machine-learning-models-e43c02325535>
- [38] Tracy Mayor. 2021. “Fintech, explained”. *MIT SLOAN*. 4 febrero, 2021. <https://mitsloan.mit.edu/ideas-made-to-matter/fintech-explained>
- [39] Upasana. 2021. “Imbalanced Data : How to handle Imbalanced Classification Problems”. *Analytics Vidhya*. Última modificación 6 enero, 2021. <https://www.analyticsvidhya.com/blog/2017/03/imbalanced-data-classification>
- [40] Zhou Xu. 2012. “Understanding Sigmoid, Logistic, Softmax Functions, and Cross-Entropy Loss (Log Loss) in Classification Problems”. *Towards Data Science*. Marzo 2012. <https://towardsdatascience.com/understanding-sigmoid-logistic-softmax-functions-and-cross-entropy-loss-log-loss-dbbbe0a17efb>

Apéndice A

Anexos

A.1. Sobre los datos de estudio

A.1.1. Extracto del diccionario de datos

Figura A.1. Extracto parte 1

Diccionario de Datos				
#	Variable	Tipo	Descripción	Fuente
1	loan_request_id	Texto (ID)	Identificador único por solicitud de crédito	Solicitud de crédito
2	user_state	Catégorica nominal	Estado de residencia del cliente	
3	economic_dependents	Númerica ordinal	Dependiente económicos del cliente	
4	housing_type	Catégorica nominal	Estatus de la vivienda del cliente	
5	icc	Númerica ordinal	Índice de capacidad crediticia	
6	marital_status	Catégorica nominal	Estado civil del cliente	
7	buro_score	Númerica continua	Score de Buró de Crédito	
8	total_debt	Númerica	Monto a refinanciar solicitado por el cliente	
9	user_age	Númerica discreta	Edad del cliente	
10	user_gender	Binaria	Género del cliente	
11	user_occupancy_type	Catégorica nominal	Profesión del cliente	
12	user_income	Númerica continua	Ingreso reportado por el cliente	
13	user_level_of_studies	Catégorica ordinal	Nivel de estudios del cliente	

Figura A.2. Extracto parte 2

14	monthsInBuro	Númerica discreta	Fecha de ingreso a buró	Reporte de Buró de Crédito
15	NumeroCuentas	Númerica discreta	Número de cuentas en buró	
16	NumeroSolicitudesUltimos6Meses	Númerica discreta	Número de consultas en los últimos 6 meses	
17	TotalSaldosActualesRevolventes	Númerica continua	Suma de los saldos actuales para cuentas de tipo revolvente y abiertas	
18	TotalSaldosVencidosRevolventes	Númerica continua	Suma de los saldos vencidos para cuentas de tipo revolvente y abiertas	
19	RevolvingLoCUsage	Númerica continua	Porcentaje del límite de crédito utilizado en relación con el saldo total	Estados de cuenta
20	bank	Catógórica nominal	Banco emisor de la tarjeta a refinanciar	
21	credit_card_name	Catógórica nominal	Nombre de la tarjeta a refinanciar	
22	credit_card_rate	Númerica continua	Promedio ponderado de las tasas de interés	
23	avgPayments	Númerica continua	Suma de las compras promedio	
24	avgBuys	Númerica continua	Suma de los gastos y/o pagos promedio	Solicitudes anteriores
25	totalRequests	Númerica discreta	Número de solicitudes anteriores	
26	pastMeanAmounts	Númerica continua	Promedio del monto solicitado	
27	stdIncome	Númerica continua	Desviación estandar de los ingresos reportados	
28	stdBCscore	Númerica continua	Desviación estandar del score de buró	Créditos anteriores
29	totalLoans	Númerica discreta	Número de créditos anteriores	
30	activeLoans	Númerica discreta	Número de créditos activos	
31	maxDaysLate	Númerica discreta	Máximo número de días de atraso	
32	averageLoanAmount	Númerica continua	Monto promedio de los créditos anteriores	
33	averageMonthlyPay	Númerica continua	Monto promedio del pago mensual de los créditos anteriores	Datos de conexión
34	browser_name	Catógórica nominal	Nombre del buscador desde donde se realizó la conexión	
35	os_name	Catógórica nominal	Sistema operativo del dispositivo desde donde se realizó la conexión	

A.1.2. Análisis Exploratorio de Datos (EDA)

Figura A.3. Matriz de correlación para variables numéricas

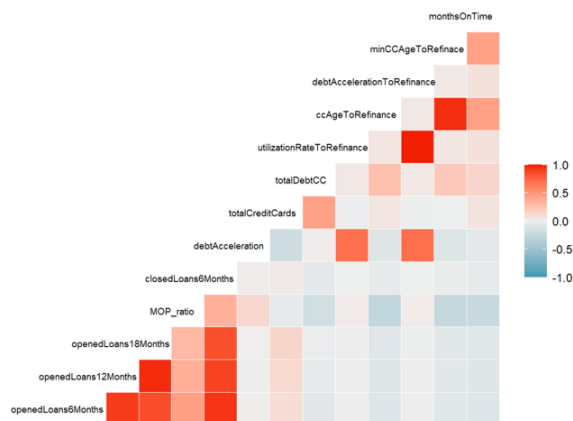


Figura A.4. Porcentajes de *default* por estado

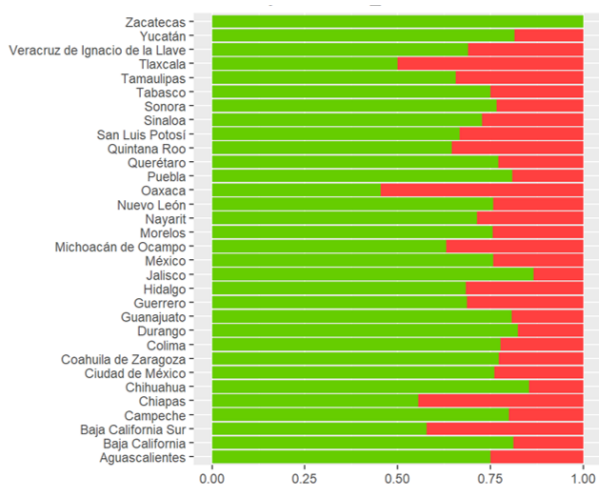


Figura A.5. Gráficas del score de Buró de Crédito

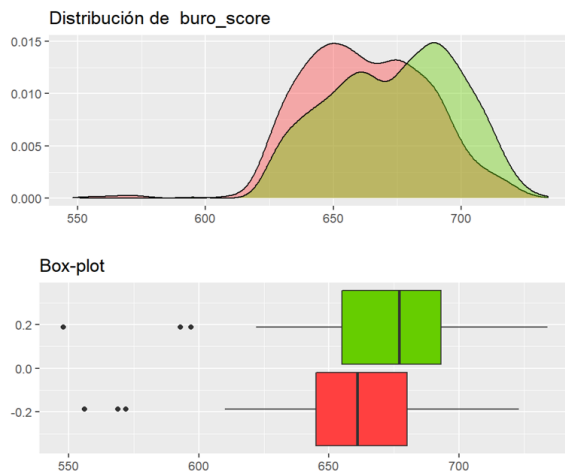


Figura A.6. Gráficas de la tasa de interés de las tarjetas a refinanciar

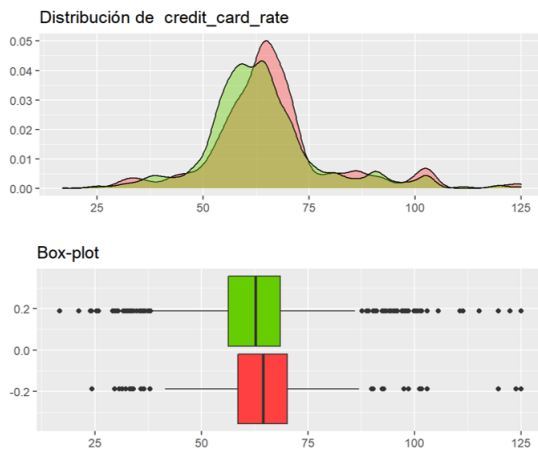


Figura A.7. Frecuencias y tasas de *default* para el número de solicitudes reportadas en Buró

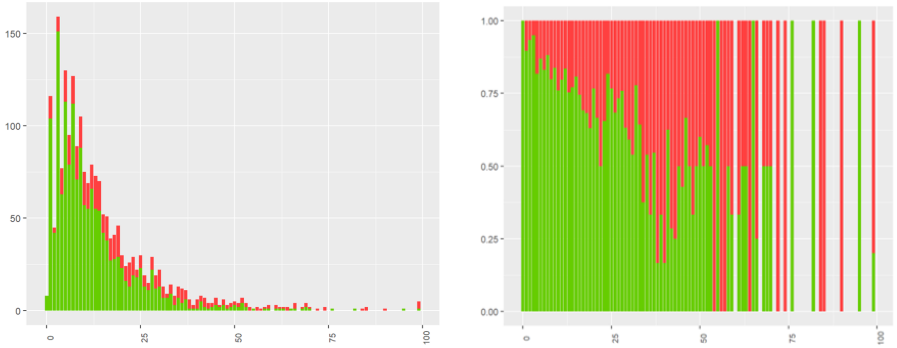
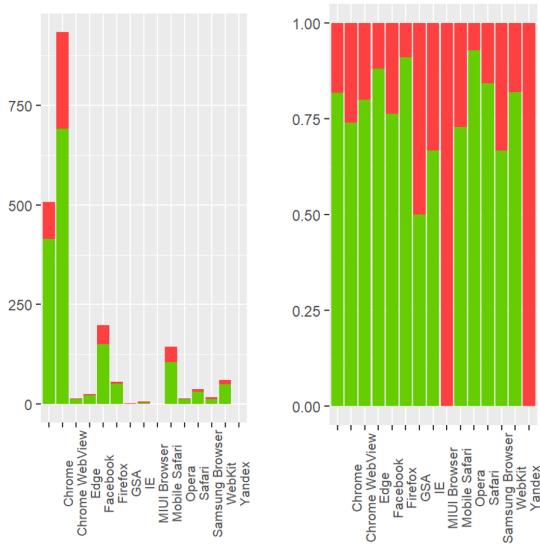


Figura A.8. Frecuencias y tasas de *default* para valores del *browser_name*



A.2. Coeficiente de correlación

El coeficiente de correlación de Pearson entre dos variables aleatorias X y Y se define como:

$$\rho_{X,Y} = \text{Corr}(X, Y) = \frac{E[(X - \mu_X) - (Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

Donde $E(X)$ es la esperanza o momento de la variable X , μ_X es el valor esperado de X y σ_X es la desviación estándar. Este coeficiente va de -1 a 1 , donde 1 significa una correlación linear positiva perfecta, -1 una correlación linear negativa perfecta y 0 independencia linear. Dado una par de observaciones (X_i, Y_i) con $i = 1, \dots, n$, el coeficiente de correlación muestral utilizado como estimación se define como:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n-1)s_X s_Y}$$

Donde $\bar{x}$ es la media muestral de X y s_x la desviación estándar muestral corregida.

A.3. WOE y IV

El *Weight of Evidence (WOE)*: es una medida del poder predictivo de una variable independiente en relación con la variable dependiente y se puede interpretar como una medida de separación entre las clases para un grupo específico de observaciones. Una vez calculados los WOE podemos introducir el segundo concepto de *Information Value (IV)*: es una métrica que nos dice qué tanta información está aportando una variable al problema de clasificación. El proceso para calcular el WOE para una variables consiste en:

1. Para variables continuas, hacer una separación de la distribución en grupos. El número de grupos a crear puede variar, pero suele

buscarse que al menos cada grupo contenga el 5 % de las observaciones, i.e. a lo más 20 grupos ¹.

2. Calcular el número de observaciones de cada clase, malos/default y buenos/no-default para cada grupo.
3. Calcular el WOE para cada grupo utilizando la siguiente definición:

$$WOE = \ln\left(\frac{\% \text{ de buenos/no-default}}{\% \text{ de malos/default}}\right)$$

Por lo tanto, obtenemos un valor del WOE para cada grupo por variable. Esta metodología también se utiliza para agrupar y transformar variables, pero esto quedará fuera del alcance de este trabajo. Una vez calculados los WOE podemos introducir el segundo concepto de *Information Value (IV)*: es una métrica que nos dice qué tanta información está aportando una variable al problema de clasificación. De manera general, el IV para una variable se define como:

$$IV = \sum (\% \text{ de buenos} - \% \text{ de malos}) * WOE$$

La suma va sobre los grupos creados en el proceso del WOE para cada variable. Esta métrica nos sirve para clasificar y seleccionar variables a partir de cuáles aportan más información. Actualmente, la regla de clasificación estándar es:

¹Por los mismos motivos descritos en la sección 3.1.2 deseamos ver pocos grupos. Es decir, para variables categóricas o numéricas (ya agrupadas) con demasiados grupos buscaremos combinar categorías.

Cuadro A.1. Esquema de clasificación de acuerdo al IV

<i>Information Value</i>	Poder predictivo
Menor a 0.02	No es útil
0.02 a 0.1	Poder predictivo débil
0.1 a 0.3	Poder predictivo medio
0.3 a 0.5	Poder predictivo fuerte
Mayor a 0.5	Poder prectivo sospechoso

A.4. SMOTE

El método *Synthetic Minority Over-sampling Technique* (SMOTE) consiste en seleccionar un subconjunto de la clase minoritaria para crear un nuevo conjunto similar de forma sintética y luego agregarlo al conjunto original. Esto evita tener que duplicar observaciones, por lo tanto mitiga el posible sobre-ajuste. La creación de nuevas observaciones se realiza utilizando aprendizaje no supervisado, en particular el método de k -vecinos Cercanos. Dependiendo la cantidad de nuevas observaciones requeridas, se toman cierto número de vecinos de manera aleatoria. Las observaciones sintéticas se crean de la siguiente manera:

1. Seleccionamos una observación del conjunto minoritario.
2. Calculamos los k -vecinos más cercanos de dicha observación dentro del conjunto minoritario.
3. Para cada característica o variable de la observación y para cada vecino:
 - a) Tomamos la diferencia entre las características de dicha

observación y del vecino más cercano seleccionado. Esto genera un vector de diferencias.

- b) Multiplicamos cada diferencia por un número aleatorio entre 0 y 1, lo sumamos a la característica original, creando una nueva característica para la observación sintética.²

Este enfoque genera que las regiones de decisión sean más grandes y por ende menos específicas. De esta manera, el clasificador que utilicemos aprenderá sobre regiones más amplias para la clase minoritaria y podrá generalizar mejor, logrando así mejorar uno de los principales objetivos.

Los algoritmos de clasificación intentan aprender las regiones de decisión, en particular la línea o el límite (*borderline*) que separa una clase de otra. Por este motivo, las observaciones que están en este límite o cerca de él son más difíciles de clasificar y por ende más importantes para el problema. A raíz de esto, surge una variación al método anterior llamada *Borderline-SMOTE*, el cual sólo crea observaciones sintéticas cerca de los límites. Entonces, la única diferencia recae en la selección de las observaciones sobre los límites y estos pasos extras consisten en:

1. Para cada observación p_j en el conjunto minoritario, calculamos sus m -vecinos más cercanos sobre el conjunto completo de observaciones (positivos y negativos). Denotamos m' como el número de vecinos que pertenecen a la clase mayoritaria.

²Notemos que si una de las variables no es continua, habría que definir una regla o tratamiento especial para calcular la diferencia. Por esta razón se creó una variación del método llamada SMOTE-NC, Se pueden encontrar más detalles en las referencias.

2. Si $m' = m$, i.e. todos los m -vecinos cercanos pertenecen a la clase mayoritaria, entonces p_j se considera ruido y no será utilizada. Si $\frac{m}{2} \geq m' < m$, entonces p_j será considerada riesgosa, pues se puede clasificar mal fácilmente. Por último, si $0 \leq m' < \frac{m}{2}$, entonces p_j es segura y tampoco será utilizada.
3. Seleccionamos sólo el conjunto de observaciones riesgosas y procedemos a aplicar *SMOTE* tal como se describió más arriba.

Mostramos una imagen para ejemplificar las regiones de decisión para un problema de clasificación binario y cómo estaría trabajando el método descrito.

Figura A.9. Regiones de decisión e ilustración del *Borderline-SMOTE* en acción

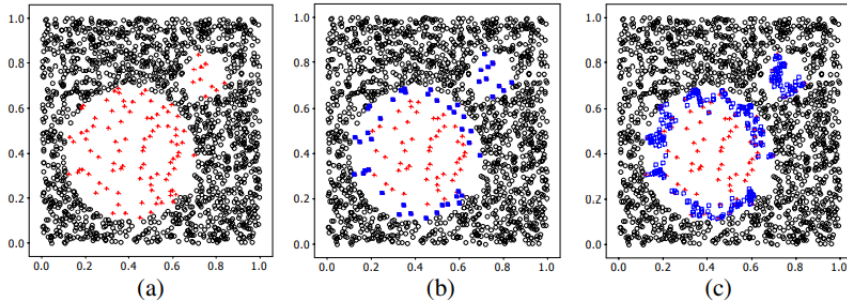


Fig. 1. (a) The original distribution of Circle data set. (b) The borderline minority examples (*solid squares*). (c) The borderline synthetic minority examples (*hollow squares*).

Fuente: Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning

A.5. Conceptos clave detrás del aprendizaje de máquina

A.5.1. Suma residual de cuadrados y mínimos cuadrados

A partir de la discrepancia entre la función lineal $f(X)$ y el valor real de la variable Y definimos la función de pérdida al cuadrado como $l(f(x, y)) = (f(x) - y)^2$ para medir la magnitud del error. De ahí, la suma residual de cuadrados (*residual sum of squares*) se define como:

$$RSS(\beta) = \sum_{i=1}^N (y_i - f(x_i))^2 = \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^k \beta_j X_{ij})^2$$

Mínimos cuadrados (*least squares*) es el método estándar para minimizar esta función y obtener los parámetros óptimos de la función lineal. Para lograr una mejor representación de este concepto vamos a presentar una imagen de esta técnica en acción para un ejemplo con $X \in \mathbb{R}^2$.

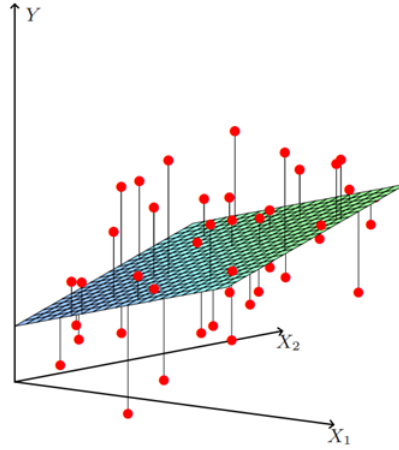


FIGURE 3.1. *Linear least squares fitting with $X \in \mathbb{R}^2$. We seek the linear function of X that minimizes the sum of squared residuals from Y .*

Fuente: Imagen tomada de la sección 3.2 del libro [14]

A.5.2. Error cuadrático medio

A partir de la suma residual al cuadrado, la función de riesgo empírico llamada Error Cuadrático Medio (*Mean Squared Error* o ERM) se define como:

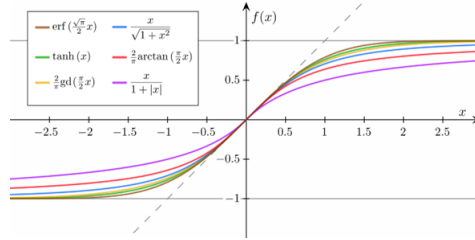
$$L_s(f) = \frac{1}{k} \sum_{i=1}^k (f(x_i) - y_i)^2$$

Esta mide el promedio de los errores al cuadrado. Es la misma función que buscamos minimizar en el problema de optimización

A.5.3. Funciones sigmoide

Las funciones sigmoide son funciones matemáticas que comparten ciertas propiedades, la más destacada es que tienen una curva de la forma S como se ve en la siguiente imagen.

Figura A.10. Miembros de la familia de funciones sigmoide



Fuente: Extraída desde Wikipedia

La función sigmoide más comúnmente utilizada en el contexto de aprendizaje de máquina es la función logística que tiene la siguiente forma.

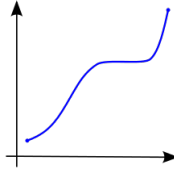
$$f(x) = \frac{1}{1 + \exp(-x)} : \mathbb{R} \rightarrow [0, 1]$$

Esta tiene algunas propiedades muy importantes como que mapea el universo de parámetros en una función de probabilidad, utiliza la función exponencial, es convexa y diferenciable. Se puede consultar el artículo [40] para continuar con el estudio de estos conceptos y su relación con los problemas de clasificación dentro del aprendizaje de máquina.

A.5.4. Funciones monótonas

Una función que mantiene o invierte el orden dado por el conjunto que la genera. Bajo el contexto de cálculo, una función se considera monótona creciente si $x \leq y \rightarrow f(x) \leq f(y)$ y monótona decreciente si $x \leq y \rightarrow f(x) \geq f(y)$.

Figura A.11. Ejemplo de una función monótona creciente



Fuente: Extraída desde Wikipedia

Para quien esté interesado en profundizar más en el tema se puede consultar el libro de *Thomas' Calculus* de Hass, Heil y Weir.

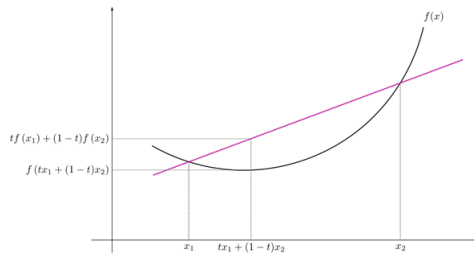
A.5.5. Funciones convexas

El concepto de convexidad se aplica tanto en conjuntos como en funciones. Una función es convexa si su dominio es un conjunto convexo y el segmento de recta que une dos puntos en su gráfica está por encima de dicha gráfica en ese segmento. Más formalmente, la segunda propiedad se define como:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \forall \alpha \in [0, 1]$$

Es más fácil entender este concepto después de observar un ejemplo.

Figura A.12. Ejemplo de una función convexa



Fuente: Extraída desde Wikipedia

Los ejemplos más clásicos de funciones convexas son la función cuadrática x^2 o la función exponencial e^x . Este concepto juega un papel fundamental en el contexto de problemas de optimización. Uno de los más relevantes es el de asegurar la existencia de un mínimo o máximo único. Para quien esté interesado en profundizar más en el tema se puede consultar el libro de *Numerical Optimization* de Jorge Nocedal y Stephen J. Wright [21].

A.5.6. Algoritmos de optimización

Los algoritmos de optimización utilizados en este contexto son iterativos, se comienza con una estimación inicial de x y se van generando mejoras en las estimaciones en cada iteración hasta terminar, deseando llegar a una solución aceptable. Una de las principales diferencias entre algoritmos es el criterio o estrategia que se usa para pasar de la iteración actual a la siguiente. Además de estas características, buscamos algunas propiedades deseables en los algoritmos o métodos de optimización:

- Robustez. Esperamos que tengan un buen desempeño en problemas distintos.
- Eficiencia. Esperamos que no requieran demasiado tiempo o memoria en términos computacionales.
- Precisión. Esperamos que obtengan una solución precisa, sin ser tan sensibles a errores en los datos o problemas de precisión aritmética asociados a la implementación computacional.

Sin embargo la intersección de estas propiedades puede presentar conflictos. Por ejemplo, un método robusto puede llevar mucho tiempo de procesamiento. En general, buscaremos un balance entre estos

objetivos, estos son algunos de los problemas centrales en la rama de optimización numérica. Esta explicación se tomó del mismo libro de *Numerical Optimization* [21], el cual es ampliamente recomendado para profundizar en el tema.

A.6. Detalles de implementación de los algoritmos

A.6.1. Regresión Logística

Dentro del universo de hiper-parámetros, vamos a concentrarnos en tres. Primero vamos a considerar la norma L2 y el caso sin penalización como posibles penalizaciones, esto nos permite probar cualquier método con o sin regularización. Por otro lado, para el inverso de la fuerza de regularización C vamos a tomar los valores 0.001, 0.009, 0.01, 0.09, 0.1, 0.9, usaremos estos mismos valores para todos los métodos que usen regularización o tasa de aprendizaje para homologar condiciones. Por último, probaremos tres algoritmos de optimización numérica, el algoritmo de Netwon, el algoritmo de Broyden-Fletcher-Goldfarb-Shanno y gradiente estocástico promedio. Utilizando la notación de scikit-learn, el espacio quedó definido como:

- $C = [0.001, 0.009, 0.01, 0.09, 0.1, 0.9]$
- $penalty = ['l2', 'none']$
- $solver = ['newton-cg', 'lbfgs', 'sag']$

Después de realizar la búsqueda de parámetros óptimos nos quedamos con $C = 0.009$, $penalty = 'l2'$ y $solver = 'newton-cg'$.

En general Se definieron conjuntos de hiper-parámetros buscando cubrir un espacio amplio de exploración, incentivado por la buena velocidad de entrenamiento. En nuestro caso, esto sucede principalmente por que contamos con un conjunto de datos “pequeño”.

A.6.2. Regresión Ridge

Dentro del universo de hiper-parámetros, vamos a concentrarnos en dos. Primero el poder de penalización *alpha* será el mismo conjunto utilizado para su inverso *C* en la regresión logística. Además, pondremos a prueba varios algoritmos de optimización, casi todos distintos a los antes mencionados, para generar mayor contraste.

- *alpha* = [0.001, 0.009, 0.01, 0.09, 0.1, 0.9]
- *solver* = ['auto', 'svd', 'cholesky', 'lsqr', 'sparse_cg']

Después de realizar la búsqueda de parámetros óptimos nos quedamos con *alpha* = 0.9 y *solver* = 'lsqr'.

A.6.3. XGBoost

Dentro del universo de hiper-parámetros, vamos a concentrarnos en cuatro. Primero, especificamos *gamma* que corresponde a la reducción logarítmica mínima requerida para hacer una partición en un nodo del árbol. Segundo, la tasa de aprendizaje denotada por *learning_rate*. Tercera, la profundidad máxima del árbol denotado por *max_depth*. Cuarta, la suma mínima del peso de la instancia requerida en cada hijo del árbol, denotada por *min_child_weight*.

- *gamma* = [0, 1, 5]
- *learning_rate* = [0.001, 0.009, 0.01, 0.09, 0.1, 0.9]

- $max_depth = [1, 2, 5, 6, 9, 10]$
- $min_child_weight = [0.5, 1, 2, 5]$

Después de realizar la búsqueda de parámetros óptimos nos quedamos con $gamma = 0$, $learning_rate = 0.09$, $max_depth = 2$ y $min_child_weight = 2$.

A.6.4. LGBM

Dentro del universo de hiper-parámetros, vamos a concentrarnos en tres. Primero, tenemos la tasa de aprendizaje denotada por $learning_rate$. Segundo, el mínimo de observaciones por hoja del árbol denotado por $min_data_in_leaf$. Tercero, el máximo número de hojas por árbol denotado por num_leaves .

- $learning_rate = [0.001, 0.009, 0.01, 0.09, 0.1, 0.9]$
- $min_data_in_leaf = [10, 20, 40]$
- $num_leaves = [5, 10, 20]$

Después de realizar la búsqueda de parámetros óptimos nos quedamos con $learning_rate = 0.1$, $min_data_in_leaf = 20$ y $num_leaves = 5$.

Índice de cuadros

3.1. Distribución final de los conjuntos de datos	16
3.2. Ejemplos de variables con datos faltantes	18
3.3. Ejemplos de los distintos tratamientos para datos faltantes	19
3.4. Tipo de variables y codificación a utilizar	23
3.5. Nueva agrupación para <i>user_state</i>	25
3.6. Métricas por método de selección	37
3.7. Votos por método de selección	37
3.8. Algoritmos de clasificación	40
4.1. Resultados aplicando la codificación sin agrupar clases .	66
4.2. Resultados aplicando la codificación después de agrupar clases	67
4.3. Resultados aplicando estandarización	68
4.4. Resultados aplicando la mezcla de transformaciones . .	69
4.5. Resultados aplicando el método de votación para la selección de variables	70
4.6. Resultados óptimos hasta el momento	71
4.7. Resultados aplicando <i>Random Over Sampling</i>	72
4.8. Resultados aplicando SMOTE	72
4.9. Resultados aplicando <i>Borderline</i> -SMOTE	73
4.10. Resultados después de actualizar el conjunto de datos .	74

4.11. Resultados finales del preprocesamiento óptimo	75
4.12. Resultados con hiper-parámetros óptimos	76
4.13. Variables más y menos relevantes	81
4.14. Distribución del conjunto de prueba por zona	91
4.15. Distribución del conjunto de prueba y rechazo	92
A.1. Esquema de clasificación de acuerdo al IV	109

Índice de figuras

3.1. Distribución y transformaciones de la variable <i>user_income</i>	31
3.2. Distribución y transformaciones de la variable <i>DebtToIncome</i>	32
3.3. Gráfica de la función logística	43
3.4. Ejemplo de un árbol de decisión	48
3.5. Ejemplo de una matriz de confusión	55
3.6. Ejemplos de curvas ROC	58
3.7. Representación visual de la validación cruzada	61
4.1. Distribuciones del <i>score</i> de cada modelo	78
4.2. Coeficientes más influyentes para las regresiones	79
4.3. Variables más influyentes para los modelos de ensamble	80
4.4. Comparación de los <i>scores</i> de los modelos	82
4.5. Zona de aprobación para cada modelo	85
4.6. Zona de rechazo para cada modelo	86
4.7. Comparación de los dos <i>scores</i> finales	88
4.8. Matriz de contingencia del <i>default</i> acumulado	89
A.1. Extracto parte 1	102
A.2. Extracto parte 2	103
A.3. Matriz de correlación para variables numéricas	104

A.4. Porcentajes de <i>default</i> por estado	104
A.5. Gráficas del score de Buró de Crédito	105
A.6. Gráficas de la tasa de interés de las tarjetas a refinanciar	105
A.7. Frecuencias y tasas de <i>default</i> para el número de solicitudes reportadas en Buró	106
A.8. Frecuencias y tasas de <i>default</i> para valores del <i>browser_name</i>	106
A.9. Regiones de decisión e ilustración del <i>Borderline-SMOTE</i> en acción	111
A.10. Miembros de la familia de funciones sigmoide	114
A.11. Ejemplo de una función monótona creciente	115
A.12. Ejemplo de una función convexa	115

*Diseño, Construcción y Validación
de un Modelo de Riesgo Crediticio*

escrito por Santiago Muriel,

se terminó de imprimir en noviembre de 2022

en los talleres de Tesis Matoso.

Campeche 156, colonia Roma,

Ciudad de México.